Análisis y Diseño de Algoritmos

Dr. José Martínez Carranza

Otoño 2026 - Bryan Ezequiel Violante Arriaga

Índice

1. Introducción al Curso **2**

2. Funciones asintóticas **6**

3. Continuación de funciones asintóticas **14**

4. Anexo: caracterización por límites **20**

5. Recurrencias **23**

6. Métodos para resolver recurrencias **25**

7. Heaps **32**

8. Mochila fraccional, actividades y divide y vencerás **37**

9. Multiplicación de enteros grandes **41**

10. Programación dinámica **45**

11. Grafos **54**

1. Introducción al Curso

18 de agosto de 2026

Resumen rápido. Sesión inicial: temario, criterios de evaluación y datos de contacto. El profesor comparó bubble sort contra quick sort a partir del video de Obama y de ahí saltó a por qué sigue valiendo la pena estudiar estructuras de datos y algoritmos en la era de la IA. Cerró con computable, decidible e indecidible, y con P contra NP.

Info general

Temario:

1. Introducción
2. Notación asintótica
3. Recurrencias
4. Ordenamiento
5. Algoritmos voraces
6. Divide y vencerás
7. Programación dinámica
8. *(falta anotar este tema)*
9. Algoritmos aleatorizados
10. Teoría de complejidad

Criterios de evaluación:

- 90 % exámenes, tres parciales de 30 % cada uno
- 10 % asistencia

Contacto y material:

- Sitio del curso: `http://ccc.inaoep.mx/~carranza/alg.html`
- Correo: `carranza@inaoep.mx`

Ojito...

Todo correo al profesor lleva `ALG2026` en el asunto. Si no, se pierde en el SPAM.

El objetivo del curso es desarrollar pensamiento computacional para resolver problemas. No se trata tanto de programar sino de analizar y diseñar algoritmos eficientes.

Ordenamiento (por arribita)

A raíz del vídeo de Obama, en el cual le preguntan cómo ordenar un millón de enteros, se da una embarradita de algoritmos de ordenamiento. Son los dos ejemplos con los que el profesor mostró que dos algoritmos correctos para el mismo problema no cuestan lo mismo.

Bubble sort

La idea es ir burbujeando los elementos. Se recorre el arreglo comparando pares vecinos y se intercambian cuando están en desorden, así que en cada pasada el mayor de los que quedan sube hasta su posición final. Se caracteriza por usar 2 ciclos anidados, lo que denota su complejidad cuadrática.

Mejor caso, caso promedio y peor caso son los tres $\mathcal{O}(n^2)$. La versión simple recorre los dos ciclos anidados completos sin importar cómo venga el arreglo, así que un arreglo ya ordenado le cuesta lo mismo que uno al revés.

Quick sort

Parte el problema. Elige un pivote, acomoda a la izquierda lo menor y a la derecha lo mayor, y se llama a sí mismo sobre cada mitad. Como las dos mitades ya no dependen una de la otra, el algoritmo es paralelizable.

Algoritmo	**Mejor**	**Promedio**	**Peor**	**Memoria**
Bubble sort	$\mathcal{O}(n^2)$	$\mathcal{O}(n^2)$	$\mathcal{O}(n^2)$	$\mathcal{O}(1)$
Quick sort	$\mathcal{O}(n \log n)$	$\mathcal{O}(n \log n)$	$\mathcal{O}(n^2)$	$\mathcal{O}(\log n)$

Tabla 1: Costo de los dos algoritmos vistos en clase.

Idea clave

Dos algoritmos igual de correctos para el mismo problema pueden diferir en órdenes de magnitud. Escoger el algoritmo es una decisión de diseño, no un detalle de implementación.

Por qué estudiar DSA

Saber como funcionan los algoritos y como usar las estructuras de datos nos permite

- Romper problemas en pasos pequeños.
- Elegir la estructura de datos correcta.
- Analizar la complejidad en tiempo y en memoria, porque siempre se busca la solución óptima.
- Comparar diseños y sus trade-offs.

Se espera que al terminar el curso uno pueda determinar la estrategia correcta para resolver un problema y dar soluciones óptimas, escalables y eficientes en tiempo y espacio.

Conceptos clave

- **Algoritmo:** secuencia finita y no ambigua de pasos que, a partir de una entrada, produce una salida en un número finito de operaciones.
- **Problema computable:** el problema para el que sí se puede estructurar un programa que obtenga un resultado. Saber cuál es el último número natural no lo es: no hay tal número, así que no hay programa que lo devuelva.
- **Problema decidible:** existe un algoritmo que, para cualquier entrada, se detiene y entrega una respuesta.
- **Problema indecidible:** no existe tal algoritmo. Ninguna solución general lo resuelve.

Verificación

Si desarrollo un algoritmo que va a resolver un problema, la pregunta inmediata es cómo sé que esa solución es válida. Esa es la verificación, y es un paso aparte de escribir el código.

DSA en tiempos de la IA

Conocer estructuras de datos y algoritmos sirve para analizar el código que la IA genera. El vibe coding sirve para prototipar rápido, pero *abstraction eventually leaks*: tarde o temprano hay que bajar al detalle que la abstracción escondía. Los dos sirven mejor en conjunto.

Ojito...

El código generado siempre lo juzga un humano, para detectar áreas de oportunidad y cosas por optimizar. Sin ese criterio no hay forma de saber si lo que salió es bueno.

Cómo convertirse en experto en DSA

Antes era a la antigua: tirar código hasta que la intuición apareciera sola y uno supiera qué estructura de datos usar en cada caso. Eso ya cambió. Ahora se aprende la teoría primero, se entienden los fundamentos, se estudian las matemáticas detrás de DSA y luego se practica con ejemplos.

El ciclo de trabajo que espera el profesor:

1. Intentarlo por mi cuenta.
2. Pedir una pista, no la solución.

3. Implementar y probar.
4. Preguntarle a la IA por mi razonamiento.
5. Explicarle la solución a la IA.
6. Resolver un problema similar sin IA.

Idea clave

La práctica hace al maestro. La IA entra como quien revisa, no como quien resuelve.

El santo grial: P contra NP

P es la clase de los problemas que se resuelven en tiempo polinomial. NP es la de los problemas cuya solución propuesta se *verifica* en tiempo polinomial, aunque encontrarla parezca costar tiempo exponencial. Todo problema de P está en NP, porque resolverlo ya es una forma de verificarlo. La pregunta abierta es la contraria: si algo se verifica rápido, ¿también se resuelve rápido?

Demostrar $P = NP$ implicaría que miles de problemas que hoy se atacan por aproximación tienen algoritmo eficiente y nadie lo ha encontrado. Demostrar $P \neq NP$ cerraría la puerta de una vez. Nadie ha hecho ninguna de las dos.

La IA agentica vamos para allá... (La IA que resuelve problemas por sí misma, es decir, puede fijar metas, planificar y ejecutar tareas de forma humana para alcanzar un objetivo específico, con poca supervisión humana) pero eso nos podría hacer tratar todo como una caja negra... pero eso no es lo ideal. Siempre queremos entender cómo funcionan las cosas, y el estudio de DSA nos da esas herramientas.

2. Funciones asintóticas

20 de agosto de 2026

Resumen rápido. En esta clase vimos una intro a la notación asintótica y a las funciones de crecimiento, asi como un ejemplo/tecnica de como se puede analizar la complejidad de un algoritmo. Finalizamos la clase con un poco de histria, hablando sobre el Debate Lighthill de 1973, "The general purpose robot is a mirage"

Qué es una función?

Definición 2.1. Una función es una relación entre dos conjuntos, donde a cada elemento del primer conjunto (*dominio*) le corresponde un único elemento del segundo conjunto (*codominio*). conjunto.

COnceptos importantes:

- Dominio: conjunto de entrada, todos los valores de entrada permitidos, por ejemplo, los reales.
- Codominio: conjunto de llegada, donde viven las salidas.
- Imagen: conjunto de llegada que realmente se obtiene. SUbconjunto del codominio.

Ojito...

Rango y recorrido son sinónimos de imagen.

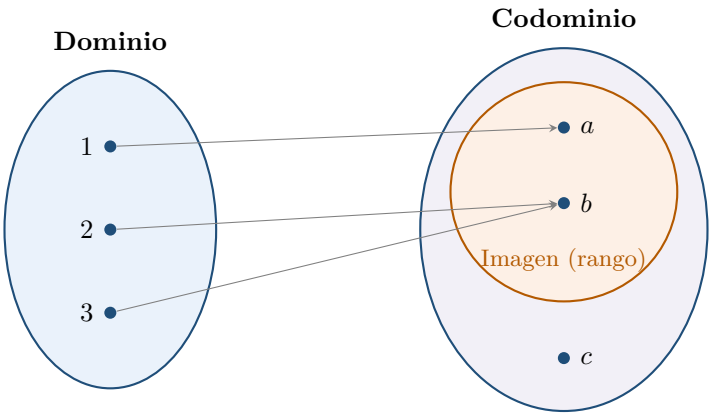


Figura 1: El elemento c pertenece al codominio pero no a la imagen.

Una funcion de los reales a los reales por ejemplo se denota con

$$f : \mathbb{R} \to \mathbb{R}$$

que basicamente dice que es una funcion que toma un real y devuelve un real. Si esta funcion se define como $f(x) = x^2$, entonces el dominio es $\mathbb{R}$, el codominio es $\mathbb{R}$ y la imagen es $\mathbb{R}_{\geq 0}$.

Analisis Asintotico

Supongamos que tenemos un arreglo con n elementos desordenados y queremos ordenarlos, como ya sabemos tenemos varias opciones que hacen la misma tarea, pero se comportaran diferente dependiendo del tamaño del arreglo.

Ordenamiento Burbuja

El arreglo va creciendo y como se observa en la grafica de abajo, el tiempo que tarda en ordenarse tambien

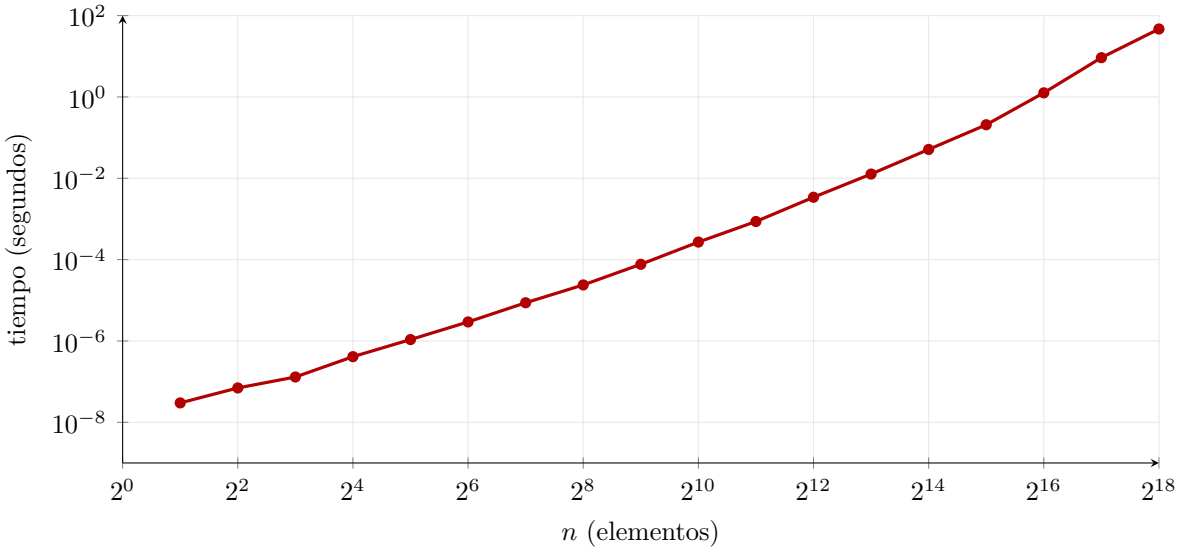


Figura 2: Tiempo de bubble sort sobre arreglos aleatorios de 2^1 a 2^{18} elementos.

Estos datos son discretos, no son continuos pero podemos aproximar una funcion, interpolar, usar alguna tecnica de algoritmos evolutivos o hasta con intuicion

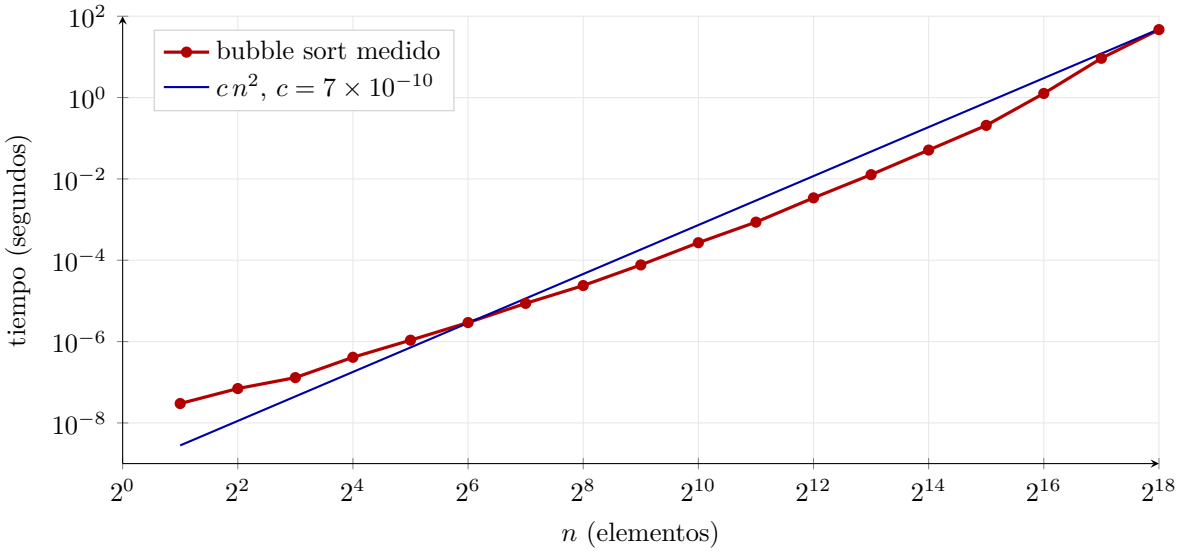


Figura 3: Tiempos medidos de bubble sort contra la curva $c\,n^2$.

Se puede ver que la curva $c\,n^2$ está encima de los puntos medidos, así que podemos decir que el tiempo de bubble sort es $\mathcal{O}(n^2)$.

Pendiente por resolver

La curva $c\,n^2$ es una cota superior, pero no es la mejor. ¿Cómo encontrar la mejor cota?... LO digo pq como que no acota todos los puntos y ademas se ve como una linea recta pero quiza sea por la escala
UPDATE: No toca los puntos por lo dicho abajo, para valores pequeños es irrelevante, el problema es que no se eligio un buen n_0 pero le pregunto al profe.

Quick sort

Ahora el mismo experimento con quick sort, los datos (resultados) se muestran a continuacion:

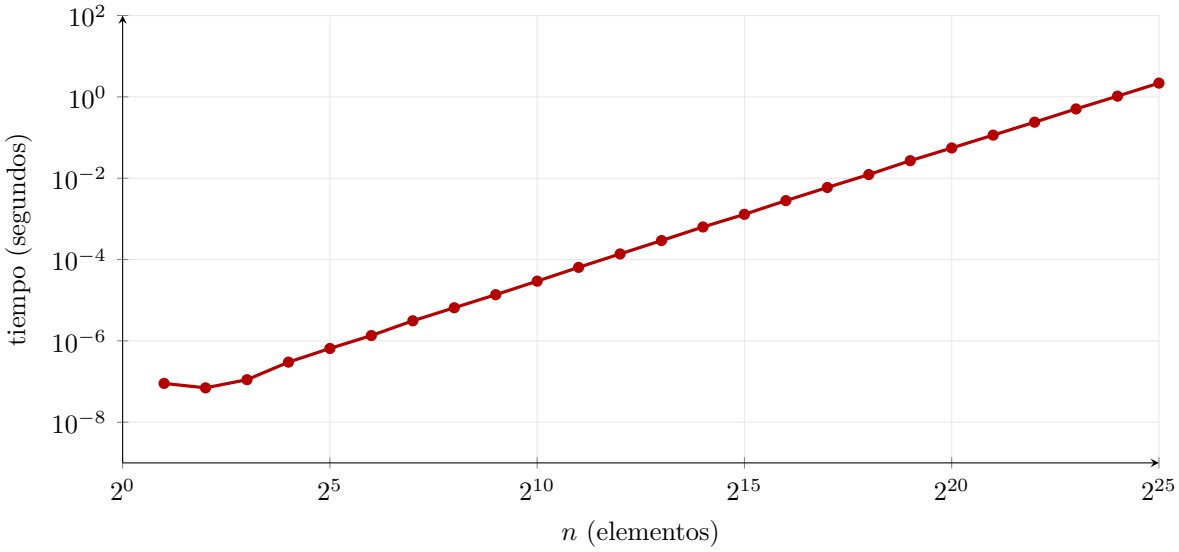


Figura 4: Tiempo de quick sort sobre arreglos aleatorios de 2^1 a 2^{25} elementos.

Al tratar de determianr la funcion analitica a ojo de buen cubero...

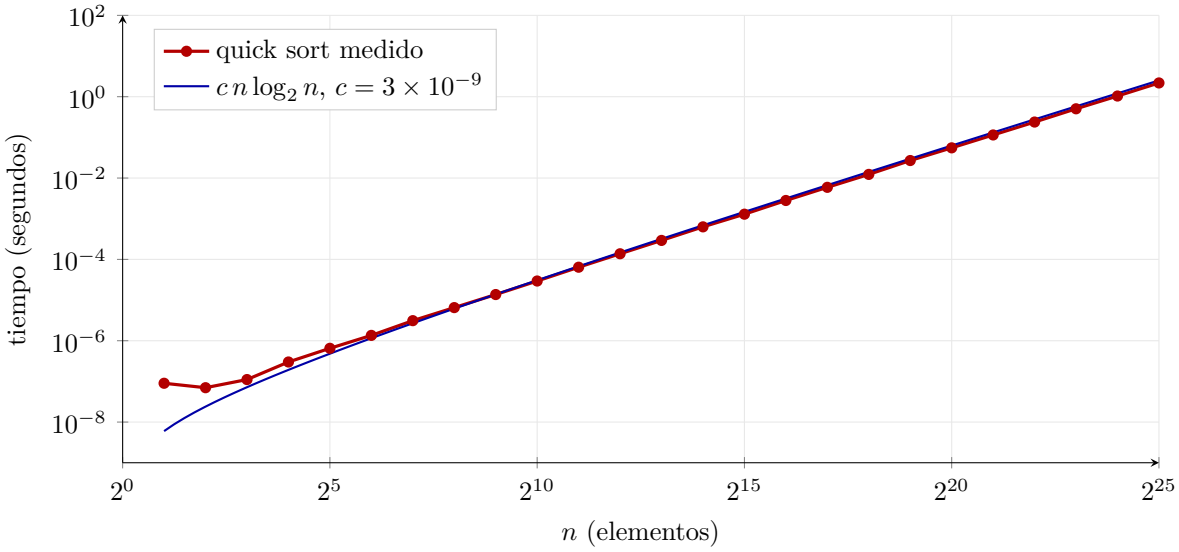


Figura 5: Tiempos medidos de quick sort contra la curva $c\,n\log_2 n$.

La curva $c\,n\log_2 n$ acota mas o menos bien los puntos medidos, asi que podemos decir que el tiempo de quick sort es $\mathcal{O}(n\log n)$.

Pendiente por resolver

Igual que en burbuja, por que no acota todos los puntos??
UPDATE: No toca los puntos por lo dicho abajo, para valores pequeños es irrelevante, el problema es que no se eligio un buen n_0 pero le pregunto al profe.

Brubuja vs Quick

Al comparar ambos algoritmos en la sig grafica...

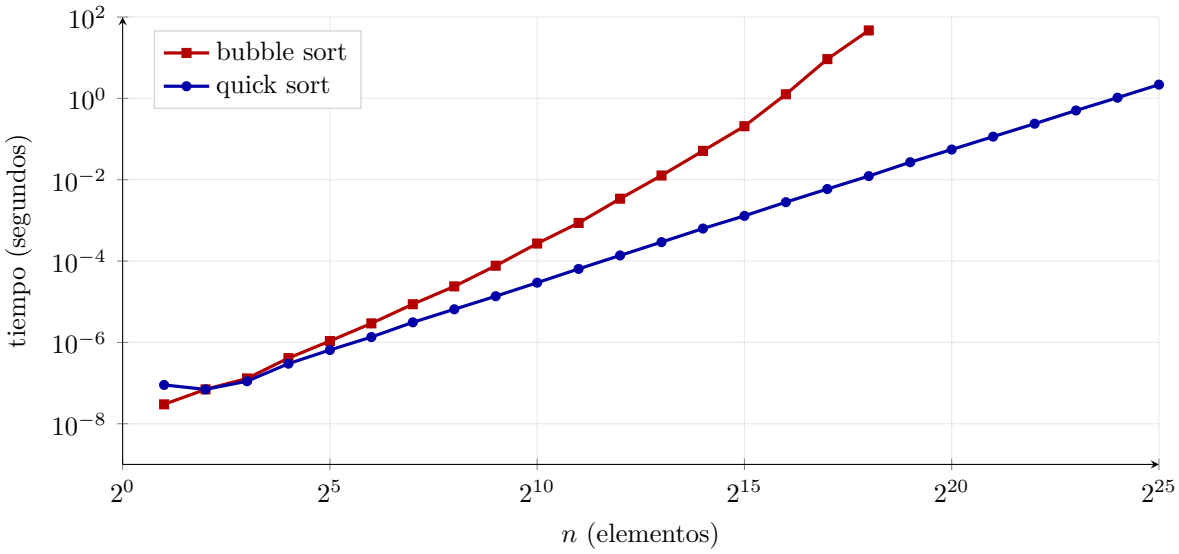


Figura 6: Bubble sort contra quick sort sobre los mismos arreglos. segundos.

Se puede ver que quick sort es mucho mas rapido que burbuja, y la diferencia se hace mas evidente a medida que el tamaño del arreglo crece. Por ejemplo, burbuja lo detuve en 2^{18} elementos porque ya tardaba demasiado, mientras que quick sort lo corrí hasta 2^{25} elementos y seguía siendo razonable. Entonces podemos concluir que quick sort es mucho mas eficiente que burbuja.

Ojito...

Se puede hacer esta comparacion ya que los arreglos que se usan para probar ambos algoritmos fueron los mismos y se testeo en la misma PC. El codigo se encuentra en https://github.com/zxxz456/inaoe/tree/main/ada/dummy/ComparaOrdenamientos.c

Notacion Big O

En los ejemplos de arriba mencione que el tiempo de bubble sort es $\mathcal{O}(n^2)$ y el tiempo de quick sort es $\mathcal{O}(n \log n)$.

Lo que quiere decir esto es que la funcion empirica encontrada del ALGORITMO de ordenamiento burbuja $f(n)$, pertenece al conjunto de funciones o a la familia de funciones $\mathcal{O}(n^2)$

Y analogamente la funcion empirica encontrada del ALGORITMO de ordenamiento quick sort $f(n)$, pertenece al conjunto de funciones o a la familia de funciones $\mathcal{O}(n \log n)$

De forma mas general, nuestra funcion empirica $f(n)$ la queremos modelar como una funcion continua $c \cdot g(n)$ y dada esta funcion $g(n)$ denotamos $\mathcal{O}(g(n))$ al conjunto de funciones tales que existen constntes positivas c y n_0 tales que para todo $n \geq n_0$ se cumple que $f(n) \leq c \cdot g(n)$

Definición 2.2. Sea $g(n)$ una función positiva. Decimos que una función $f(n)$ pertenece a $\mathcal{O}(g(n))$ si existen constantes positivas c y n_0 tales que

$$0 \leq f(n) \leq c \cdot g(n) \qquad \text{para todo } n \geq n_0.$$

Equivalentemente,

$$\mathcal{O}(g(n)) = \{\, f(n) : \exists\, c > 0,\ \exists\, n_0 > 0 \text{ tales que } 0 \leq f(n) \leq c \cdot g(n) \ \ \forall\, n \geq n_0 \,\}.$$

Y lo que dice basicamente es que $f(n)$ esta en el conjunto de funciones $\mathcal{O}(g(n))$ y que crece a lo mucho como esa $g(n)$

Ejemplo 2.3. Sean $f(n) = 2n^2 + 3n + 1$ y $g(n) = n^2$. Tomando $c = 6$ y $n_0 = 1$ se cumple

$$2n^2 + 3n + 1 \ \leq\ 6n^2 \iff 4n^2 - 3n - 1 \geq 0 \iff (4n+1)(n-1) \geq 0,$$

lo cual es cierto para todo $n \geq 1$. Por lo tanto $f(n) \in \mathcal{O}(n^2)$.

El par (c, n_0) no es único: con $c = 3$ la desigualdad $2n^2 + 3n + 1 \leq 3n^2$ equivale a $n^2 - 3n - 1 \geq 0$, que se satisface a partir de $n \geq \frac{3+\sqrt{13}}{2} \approx 3.30$, de modo que $n_0 = 4$ también funciona. La definición solo exige que *exista* al menos un par de constantes.

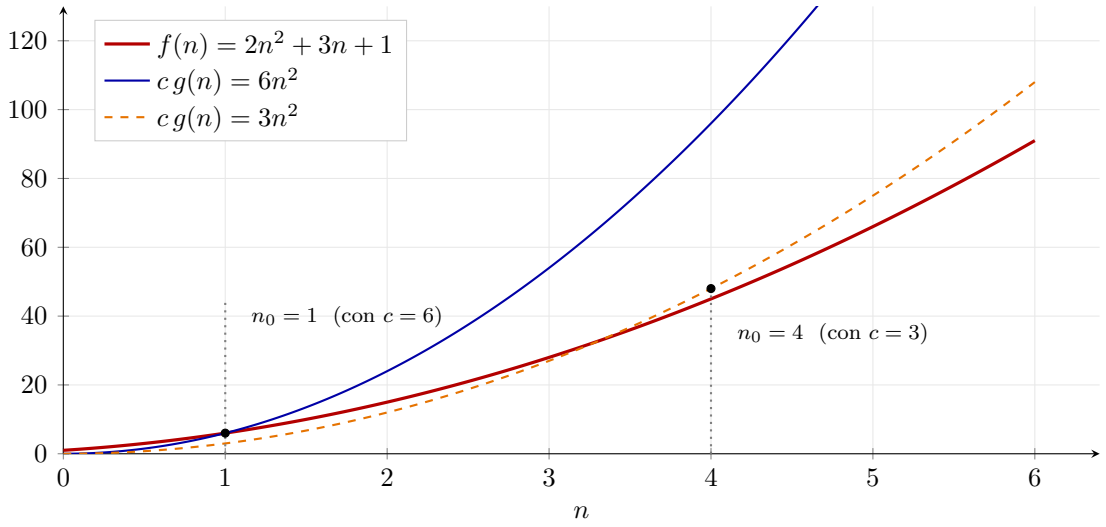


Figura 7: Para $n \geq 1$ la curva $6n^2$ domina a $f(n)$; a partir de $n \geq 4$ basta con $3n^2$.

Ojito...

El "para todo $n \geq n_0$" es lo que hace que la def sea asintotica. El comportamiento en entradas chiquitas es irrelevante, la curva f puede estar por encima de $cg(n)$ tanto como sea antes del umbral.

Ojito...

La Big O da una cota superior q no tiene que ser ajustada. Es perfectamente cierto que $n = \mathcal{O}(n^5)$, aunque sea una cota muy floja. Cuando quieres afirmar que la cota es ajustada por arriba y por abajo, usas $\Theta(g(n))$, definida con dos constantes: $0 \leq c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n)$.

Ojito...

Como $\mathcal{O}(g(n))$ es un conjunto, lo correcto sería escribir $f(n) \in \mathcal{O}(g(n))$. La costumbre de escribir $f(n) = \mathcal{O}(g(n))$ es un abuso de notación establecido, y por eso el signo no es simétrico: puedes escribir $2n^2 = \mathcal{O}(n^3)$, pero nunca $\mathcal{O}(n^3) = 2n^2$.

Crecimiento de funciones

Determinar la eficiencia asintótica de un algoritmo pasa por modelarlo: encontrar la función que describe su costo y ver contra qué se puede acotar. Muchas funciones admiten cota asintótica, y en la práctica casi todo termina comparándose contra tres familias: polinomios, logaritmos y exponenciales.

Operación elemental

Definición 2.4. Una operación elemental es aquella cuyo tiempo de ejecución está acotado por una constante que no depende del tamaño de la entrada. Una comparación, una asignación o una suma cuentan como una sola, sin importar qué tan grande sea n

Un algoritmo casi nunca ejecuta un solo tipo de operación. Sumar dos matrices de $n \times n$ hace sumas, pero también compara índices, incrementa contadores y accede a memoria. Se escoge una operación representativa, la que se ejecuta al menos tan seguido como cualquier otra, y se cuenta esa nada más: las demás quedan absorbidas por un factor constante

Principio de invarianza

Dos implementaciones del mismo algoritmo, en lenguajes distintos o sobre máquinas distintas, no tardan lo mismo, pero sus tiempos difieren a lo mucho por un factor constante. Esa es la razón de que la constante se descarte: lo que cambia al cambiar de máquina es ella, mientras que la clase asintótica es propiedad del algoritmo y sobrevive al cambio

Mejor caso, peor caso y caso promedio

El costo no depende solo de n, también de cómo venga la entrada, así que se analizan tres escenarios. Quick sort es el ejemplo de la tabla 1: en promedio cuesta $\mathcal{O}(n \log n)$, pero si el pivote cae siempre en un extremo las particiones quedan desbalanceadas y degenera a $\mathcal{O}(n^2)$. La burbuja simple no distingue, sus tres casos son $\mathcal{O}(n^2)$ porque recorre los ciclos completos sin importar cómo venga el arreglo

Idea clave

Siempre conviene optimizar. Es la única forma de llegar a los límites de lo que las arquitecturas actuales pueden dar

3. Continuación de funciones asintóticas

25 de agosto de 2026

Resumen rápido. Continuación de la clase anterior. Las propiedades de las tres notaciones, con la regla del límite que las decide de un jalón. $\Omega(\cdot)$ como cota inferior, $\Theta(\cdot)$ como las dos a la vez y el teorema que las relaciona. Cerramos con las notaciones chicas, los órdenes de complejidad y los consejos para contar operaciones elementales.

Propiedades de $\mathcal{O}(\cdot)$

Sean f, g, h, f_1 y f_2 funciones positivas.

1. Para cualquier función f se tiene $f \in \mathcal{O}(f)$.
2. $f \in \mathcal{O}(g) \Rightarrow \mathcal{O}(f) \subset \mathcal{O}(g)$.
3. $\mathcal{O}(f) = \mathcal{O}(g) \iff f \in \mathcal{O}(g)$ y $g \in \mathcal{O}(f)$.
4. Si $f \in \mathcal{O}(g)$ y $g \in \mathcal{O}(h) \Rightarrow f \in \mathcal{O}(h)$.
5. Si $f \in \mathcal{O}(g)$ y $f \in \mathcal{O}(h) \Rightarrow f \in \mathcal{O}(\text{mín}(g, h))$.
6. **Regla de la suma:** si $f_1 \in \mathcal{O}(g)$ y $f_2 \in \mathcal{O}(h) \Rightarrow f_1 + f_2 \in \mathcal{O}(\text{máx}(g, h))$.
7. **Regla del producto:** si $f_1 \in \mathcal{O}(g)$ y $f_2 \in \mathcal{O}(h) \Rightarrow f_1 \cdot f_2 \in \mathcal{O}(g \cdot h)$.
8. Si existe $\lim_{n\to\infty} \dfrac{f(n)}{g(n)} = k$, entonces según el valor de k:
   - Si $k \neq 0$ y $k < \infty$, entonces $\mathcal{O}(f) = \mathcal{O}(g)$.
   - Si $k = 0$, entonces $f \in \mathcal{O}(g)$, es decir $\mathcal{O}(f) \subset \mathcal{O}(g)$, pero se verifica que $g \notin \mathcal{O}(f)$.

Idea clave

La propiedad del límite es la herramienta práctica. En lugar de buscar c y n_0 a mano, se calcula el cociente f/g y se ve a dónde tiende: si tiende a una constante distinta de cero, las dos funciones son del mismo orden; si tiende a cero, f crece estrictamente más despacio.

Notación Omega

$f(n) = \Omega(g(n))$ quiere decir que $g(n)$ es una cota asintótica inferior de $f(n)$.

Definición 3.1. Sea $g(n)$ una función positiva. Decimos que una función $f(n)$ pertenece a $\Omega(g(n))$ si existen constantes positivas c y n_0 tales que

$$0 \leq c \cdot g(n) \leq f(n) \qquad \text{para todo } n \geq n_0.$$

Equivalentemente,

$$\Omega(g(n)) = \{\, f(n) : \exists\, c > 0,\ \exists\, n_0 > 0 \text{ tales que } 0 \leq c \cdot g(n) \leq f(n) \ \ \forall\, n \geq n_0 \,\}.$$

Ojito...

$f(n) \in \Omega(g(n))$ si y solo si $g(n) \in \mathcal{O}(f(n))$. Las dos notaciones son la misma relación vista desde cada lado.

Propiedades de $\Omega(\cdot)$

1. Para cualquier función f se tiene $f \in \Omega(f)$.
2. $f \in \Omega(g) \Rightarrow \Omega(f) \subset \Omega(g)$.
3. $\Omega(f) = \Omega(g) \iff f \in \Omega(g)$ y $g \in \Omega(f)$.
4. Si $f \in \Omega(g)$ y $g \in \Omega(h) \Rightarrow f \in \Omega(h)$.
5. Si $f \in \Omega(g)$ y $f \in \Omega(h) \Rightarrow f \in \Omega(\max(g, h))$.
6. **Regla de la suma:** si $f_1 \in \Omega(g)$ y $f_2 \in \Omega(h) \Rightarrow f_1 + f_2 \in \Omega(g + h)$.
7. **Regla del producto:** si $f_1 \in \Omega(g)$ y $f_2 \in \Omega(h) \Rightarrow f_1 \cdot f_2 \in \Omega(g \cdot h)$.
8. Si existe $\lim_{n\to\infty} \dfrac{f(n)}{g(n)} = k$, entonces según el valor de k:
   - Si $k \neq 0$ y $k < \infty$, entonces $\Omega(f) = \Omega(g)$.
   - Si $k = 0$, entonces $g \in \Omega(f)$, es decir $\Omega(g) \subset \Omega(f)$, pero se verifica que $f \notin \Omega(g)$.

Ojito...

En $\mathcal{O}(\cdot)$ la propiedad 5 usa el mínimo y la regla de la suma el máximo. En $\Omega(\cdot)$ la 5 usa el máximo y la suma da $g + h$. No son simétricas, conviene no copiarlas de memoria.

Por qué importan las cotas inferiores

Una cota inferior dice cuándo ya no se puede mejorar dentro de un modelo de cómputo. El ejemplo canónico es el ordenamiento por comparaciones: en el modelo general de comparaciones, cualquier algoritmo de ordenamiento necesita del orden de $n \log n$ comparaciones en el peor caso, o sea $\Omega(n \log n)$.

Idea clave

Si un algoritmo ya logra $\mathcal{O}(n \log n)$ y el problema tiene cota inferior $\Omega(n \log n)$, entonces ese algoritmo es asintóticamente óptimo en ese modelo. $\Omega(\cdot)$ contesta cuál es el trabajo mínimo que hay que hacer.

Notación Theta

$f(n) = \Theta(g(n))$ quiere decir que $c_2\, g(n)$ y $c_1\, g(n)$ son las cotas asintóticas de $f(n)$, superior e inferior respectivamente. Diremos que $f(n) \in \Theta(g(n))$ si $f(n)$ pertenece tanto a $\mathcal{O}(g(n))$ como a $\Omega(g(n))$.

Definición 3.2. Sea $g(n)$ una función positiva. Decimos que una función $f(n)$ pertenece a $\Theta(g(n))$ si existen constantes positivas c_1, c_2 y n_0 tales que

$$0 < c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n) \qquad \text{para todo } n \geq n_0.$$

Equivalentemente,

$$\Theta(g(n)) = \{\, f(n) : \exists\, c_1 > 0,\ \exists\, c_2 > 0,\ \exists\, n_0 > 0 \text{ tales que } 0 < c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n) \ \ \forall\, n \geq n_0 \,\}.$$

Propiedades de $\Theta(\cdot)$

1. Para cualquier función f se tiene $f \in \Theta(f)$.
2. $f \in \Theta(g) \Rightarrow \Theta(f) \subset \Theta(g)$.
3. $\Theta(f) = \Theta(g) \iff f \in \Theta(g)$ y $g \in \Theta(f)$.
4. Si $f \in \Theta(g)$ y $g \in \Theta(h) \Rightarrow f \in \Theta(h)$.
5. **Regla de la suma:** si $f_1 \in \Theta(g)$ y $f_2 \in \Theta(h) \Rightarrow f_1 + f_2 \in \Theta(\max(g, h))$.
6. **Regla del producto:** si $f_1 \in \Theta(g)$ y $f_2 \in \Theta(h) \Rightarrow f_1 \cdot f_2 \in \Theta(g \cdot h)$.
7. Si existe $\lim_{n\to\infty} \dfrac{f(n)}{g(n)} = k$, entonces según el valor de k:
   - Si $k \neq 0$ y $k < \infty$, entonces $\Theta(f) = \Theta(g)$.
   - Si $k = 0$, entonces los órdenes exactos de f y g son distintos.

Teorema 3.3. *$f(n) = \Theta(g(n))$ si y solo si $f(n) = \mathcal{O}(g(n))$ y $f(n) = \Omega(g(n))$.*

Demostración. De ida es inmediato: la definición de $\Theta(\cdot)$ contiene las dos desigualdades, así que c_2 y n_0 sirven de testigo para $\mathcal{O}(\cdot)$, y c_1 y n_0 para $\Omega(\cdot)$.

De regreso, si $f \in \mathcal{O}(g)$ existen c_2 y n_1 con $f(n) \leq c_2\, g(n)$ para $n \geq n_1$, y si $f \in \Omega(g)$ existen c_1 y n_2 con $c_1\, g(n) \leq f(n)$ para $n \geq n_2$. Tomando $n_0 = \max\{n_1, n_2\}$ las dos valen a la vez a partir de n_0, que es justo la definición de $\Theta(g(n))$. □

Una cota puede ser correcta y a la vez imprecisa

Sea $T(n) = 3n^2 + 7n + 20$. Todas estas afirmaciones son matemáticamente ciertas:

$$T(n) \in \mathcal{O}\left(n^2\right), \quad T(n) \in \mathcal{O}\left(n^3\right), \quad T(n) \in \mathcal{O}\left(n^4\right), \quad T(n) \in \mathcal{O}(2^n)\,.$$

Cada una de esas funciones se puede escalar por alguna constante positiva de modo que termine quedando por encima de $T(n)$. Por eso Big O por sí sola no identifica la verdadera tasa de crecimiento. $\Theta(n^2)$ sí: dice que el crecimiento cuadrático es el orden asintótico correcto, salvo factores constantes.

Ojito...

$\mathcal{O}(\cdot)$ no es sinónimo de peor caso, ni $\Omega(\cdot)$ de mejor caso. Son relaciones matemáticas de acotamiento. Primero se elige qué función de tiempo se está analizando y después se le aplica la notación que sea. Se puede decir $T_{\text{mejor}}(n) = \Theta(n)$, o $T_{\text{peor}}(n) = \Omega(n \log n)$, sin ninguna contradicción.

Notaciones chicas

$f(n) = o(g(n))$ quiere decir que $g(n)$ es una cota superior de $f(n)$ que no es asintóticamente justa (*tight*). Y $f(n) = \omega(g(n))$, que $g(n)$ es una cota inferior de $f(n)$ que tampoco lo es.

Definición 3.4. Sea $g(n)$ una función positiva. Decimos que una función $f(n)$ pertenece a $o(g(n))$ si para toda constante positiva c existe $n_0 > 0$ tal que

$$0 \leq f(n) < c \cdot g(n) \qquad \text{para todo } n \geq n_0.$$

Equivalentemente,

$$o(g(n)) = \{\, f(n) : \forall\, c > 0,\ \exists\, n_0 > 0 \text{ tales que } 0 \le f(n) < c \cdot g(n) \ \ \forall\, n \ge n_0 \,\}.$$

Definición 3.5. Sea $g(n)$ una función positiva. Decimos que una función $f(n)$ pertenece a $\omega(g(n))$ si para toda constante positiva c existe $n_0 > 0$ tal que

$$0 \le c \cdot g(n) < f(n) \qquad \text{para todo } n \ge n_0.$$

Equivalentemente,

$$\omega(g(n)) = \{\, f(n) : \forall\, c > 0,\ \exists\, n_0 > 0 \text{ tales que } 0 \le c \cdot g(n) < f(n) \ \ \forall\, n \ge n_0 \,\}.$$

La distinción precisa no es tanto lo justo o no justo, sino el cuantificador sobre la constante:

- $f(n) = \mathcal{O}(g(n)) \iff \exists\, c > 0,\ \exists\, n_0 :\ f(n) \le c\, g(n),\ \forall\, n \ge n_0.$
- $f(n) = o(g(n)) \iff \forall\, c > 0,\ \exists\, n_0 :\ f(n) < c\, g(n),\ \forall\, n \ge n_0.$

Con $\Omega(\cdot)$ y $\omega(\cdot)$ pasa lo mismo, volteando la desigualdad. Las grandes permiten que las dos funciones sean del mismo orden; las chicas lo prohíben.

Propiedad	$\mathcal{O}(g(n))$	$o(g(n))$
Tipo de cota superior	No estricta	Estricta
Condición sobre c	$\exists\, c > 0$	$\forall\, c > 0$
Permite el mismo orden	Sí	No
Puede ser $\Theta(g(n))$	Sí	No
Cociente $f(n)/g(n)$	Acotado basta	Tiende a 0

Tabla 2: Big O contra o chica.

El punto de vista del límite

Cuando el límite existe, el cociente deja la distinción clarísima:

$$f = o(g) \iff \frac{f(n)}{g(n)} \to 0, \qquad f = \Theta(g) \iff \frac{f(n)}{g(n)} \to L \text{ con } 0 < L < \infty, \qquad f = \omega(g) \iff \frac{f(n)}{g(n)} \to \infty.$$

$\mathcal{O}(\cdot)$ y $\Omega(\cdot)$ son relaciones más amplias: solo piden una cota por factor constante, no un límite específico. Por ejemplo $n^2 \neq o(n^2)$, porque el cociente vale 1 y no 0; en cambio $n = o(n^2)$, porque $n/n^2 = 1/n \to 0$.

Jerarquía de crecimiento

$$1 \ll \log n \ll n \ll n \log n \ll n^2 \ll n^3 \ll 2^n$$

Leída con o chica: $n = o(n \log n)$, $n \log n = o(n^2)$, $n^2 = o(n^3)$. Y al revés con omega chica: $n \log n = \omega(n)$, $n^2 = \omega(n \log n)$, $n^3 = \omega(n^2)$.

Idea clave

$o(\cdot) < \mathcal{O}(\cdot) - \Theta(\cdot) - \Omega(\cdot) < \omega(\cdot)$. Las grandes admiten crecimiento del mismo orden, las chicas lo prohíben. Esa es toda la diferencia.

Notación	Crecimiento	Tipo de cota
$o(g(n))$	Estrictamente más lento	Superior estricta
$\mathcal{O}(g(n))$	No más rápido	Superior
$\Theta(g(n))$	Del mismo orden	Ajustada
$\Omega(g(n))$	No más lento	Inferior
$\omega(g(n))$	Estrictamente más rápido	Inferior estricta

Tabla 3: Las cinco relaciones asintóticas de un vistazo.

Orden de complejidad

La familia $\mathcal{O}(f(n))$, $\Omega(f(n))$, $\Theta(f(n))$ define un orden de complejidad, y como representante del orden se elige la función $f(n)$ más sencilla de la familia. Se identifican diferentes familias:

Orden	Nombre
$\mathcal{O}(c)$	Constante
$\mathcal{O}(\log n)$	Logarítmico
$\mathcal{O}(n)$	Lineal
$\mathcal{O}(n \log n)$	Casi lineal
$\mathcal{O}(n^2)$	Cuadrático
$\mathcal{O}(n^3)$	Cúbico
$\mathcal{O}(n^c)$	Polinómico de grado c
$\mathcal{O}(2^n)$	Exponencial
$\mathcal{O}(n!)$	Factorial

Tabla 4: Órdenes de complejidad.

Consejos para identificar $f(n)$

Son los consejos para dar con la $f(n)$ que representa el número de operaciones elementales (OE) de un algoritmo.

Consejo 1: la base

Se asume que el tiempo de una OE es de orden 1, o sea que la constante c del principio de invarianza se toma como 1 por fines prácticos. El tiempo de ejecución de una secuencia de instrucciones, elementales o no, se obtiene sumando los tiempos de cada instrucción.

Instrucción `case`

Para `switch(n)` con casos $S_1, \ldots, S_k$:

$$f(n) = f(c) + \max\{f(S_1), \ldots, f(S_k)\},$$

donde $f(c)$ considera el tiempo de comparar n contra cada uno de los casos.

Instrucción `if`

Para `if C then` S_1 `else` S_2:

$$f(n) = f(C) + \max\{f(S_1), f(S_2)\}.$$

```
if (n == 0)
    for (i = 0; i < n; i++)
        r += i;
else
    r = 2;
```

Instrucción `while`

Para `while (c) { S }`:

$$f(n) = f(c) + (\#\text{iteraciones}) \cdot (f(c) + f(S)).$$

Ojito...

Las instrucciones `for`, `repeat` y `loop` son equivalentes a un `while`, así que se analizan con la misma fórmula.

Llamado a funciones no recursivas

El tiempo de una llamada $F(A_1, \ldots, A_s)$ es 1 por el llamado, más el tiempo de evaluar los parámetros, más el tiempo de ejecutar el cuerpo de la función:

$$f(A_1, \ldots, A_s) = 1 + f(A_1) + \cdots + f(A_s) + f(S).$$

Funciones recursivas

El tiempo de ejecución de una función recursiva se calcula a través de la solución de funciones de recurrencia, que es el tema que sigue.

4. Anexo: caracterización por límites

Herramienta de diagnóstico para $\mathcal{O}(\cdot)$, $\Omega(\cdot)$, $\Theta(\cdot)$, $o(\cdot)$ y $\omega(\cdot)$.

La idea que hay detrás

Todas estas notaciones comparan el tamaño de dos funciones, y la manera natural de comparar tamaños es dividir. Si f y g son positivas para n suficientemente grande, toda la información relevante está en el cociente $f(n)/g(n)$.

Traducción de cada notación al cociente

$f \in \mathcal{O}(g)$ significa que el cociente está *acotado por arriba* en la cola. $f \in \Omega(g)$ significa que está *acotado lejos de cero*. $f \in \Theta(g)$ significa ambas cosas. Las versiones minúsculas son las variantes estrictas: $o(\cdot)$ pide que el cociente *colapse* a cero, y $\omega(\cdot)$ que *explote* a infinito.

Conviene ver por qué la definición de $\mathcal{O}(\cdot)$ es exactamente un enunciado de acotación. La condición $f(n) \leq c\, g(n)$ para todo $n \geq n_0$ equivale, dividiendo entre $g(n) > 0$, a $f(n)/g(n) \leq c$ para todo $n \geq n_0$. Es decir: la constante c de la definición es literalmente una cota superior del cociente, y el umbral n_0 es el permiso para ignorar un tramo inicial finito. Esa lectura es la que hace transparente todo lo demás.

Criterio con el límite ordinario

Este es el caso cómodo y cubre la gran mayoría de los ejercicios, porque polinomios, logaritmos y exponenciales son funciones bien portadas cuyos cocientes convergen.

Supóngase que existe

$$L \;=\; \lim_{n\to\infty} \frac{f(n)}{g(n)}$$

admitiendo el valor $+\infty$. Entonces se tiene la clasificación siguiente.

Valor de L	Relación más fuerte	Relaciones implicadas
$L = 0$	$f \in o(g)$	$f \in \mathcal{O}(g)$
$0 < L < \infty$	$f \in \Theta(g)$	$f \in \mathcal{O}(g)$ y $f \in \Omega(g)$
$L = \infty$	$f \in \omega(g)$	$f \in \Omega(g)$

Tabla 5: Clasificación según el límite del cociente.

En el primer renglón fallan $\Omega(\cdot)$, $\Theta(\cdot)$ y $\omega(\cdot)$. En el segundo fallan $o(\cdot)$ y $\omega(\cdot)$. En el tercero fallan $\mathcal{O}(\cdot)$, $\Theta(\cdot)$ y $o(\cdot)$. Y si el límite no existe, el criterio no dice nada y hay que pasar a la sección siguiente.

Observación. El primer renglón es la fuente de un error frecuente. Que $L = 0$ no excluye a $\mathcal{O}(\cdot)$: como $o(g) \subsetneq \mathcal{O}(g)$, la afirmación $n \in \mathcal{O}(n^2)$ es perfectamente cierta, solo que floja. Simétricamente, $L = \infty$ no excluye a $\Omega(\cdot)$.

Criterio general: límite superior e inferior

Cuando el cociente oscila, el límite ordinario no existe y hace falta una herramienta que siempre esté definida. Esa herramienta es el par $\limsup$, $\liminf$.

Definición

Dada una sucesión a_n, para cada entero N se considera el supremo de la cola que empieza en N:

$$s_N \;=\; \sup\{\, a_n \;:\; n \geq N \,\}, \qquad i_N \;=\; \inf\{\, a_n \;:\; n \geq N \,\}.$$

Al aumentar N se le quitan elementos al conjunto, de modo que su supremo no puede crecer y su ínfimo no puede decrecer: la sucesión s_N es no creciente y la sucesión i_N es no decreciente. Toda sucesión monótona tiene límite en la recta extendida, y por eso las dos definiciones siguientes tienen sentido siempre, sin hipótesis adicionales:

$$\limsup_{n\to\infty} a_n \;=\; \lim_{N\to\infty} s_N, \qquad \liminf_{n\to\infty} a_n \;=\; \lim_{N\to\infty} i_N.$$

Lectura informal

El límite superior es el valor más alto al que la sucesión regresa infinitas veces; el límite inferior, el más bajo. Coinciden si y solo si la sucesión converge, y en ese caso su valor común es el límite ordinario.

Los criterios

Con $f, g > 0$ eventualmente y $a_n = f(n)/g(n)$:

Relación	Criterio equivalente
$f \in \mathcal{O}(g)$	$\limsup_{n\to\infty} a_n < \infty$
$f \in \Omega(g)$	$\liminf_{n\to\infty} a_n > 0$
$f \in \Theta(g)$	$\liminf_{n\to\infty} a_n > 0$ y $\limsup_{n\to\infty} a_n < \infty$
$f \in o(g)$	$\lim_{n\to\infty} a_n = 0$
$f \in \omega(g)$	$\lim_{n\to\infty} a_n = \infty$

Tabla 6: Criterios con límite superior e inferior.

Nótese la asimetría de los dos últimos renglones: ahí aparece el límite ordinario y no el superior o el inferior. La razón está en la forma de las definiciones. Las de $\mathcal{O}(\cdot)$ y $\Omega(\cdot)$ empiezan con *existe* una constante, y basta con controlar la cola por un lado, lo cual es justamente lo que mide un $\limsup$ o un $\liminf$. Las de $o(\cdot)$ y $\omega(\cdot)$ empiezan con *para toda* constante, y esa cuantificación universal obliga a que la cola entera se comprima, no solo una subsucesión. Comprimirse entera es exactamente converger.

Relaciones estructurales

Las inclusiones siguientes son útiles para no reportar una relación más débil de la que se tiene.

- $o(g) \subsetneq \mathcal{O}(g)$ y $\omega(g) \subsetneq \Omega(g)$: las minúsculas son las cotas estrictas.
- $\Theta(g) = \mathcal{O}(g) \cap \Omega(g)$.
- $f \in \mathcal{O}(g)$ si y solo si $g \in \Omega(f)$; la misma dualidad relaciona $o(\cdot)$ con $\omega(\cdot)$.
- $\mathcal{O}(\cdot)$ y $\Omega(\cdot)$ pueden fallar ambos a la vez, y entonces las funciones son incomparables.
- $o(\cdot)$ y $\omega(\cdot)$ nunca se cumplen simultáneamente.

Un ejemplo donde el límite ordinario no sirve

Sea $f(n) = n^2$ y sea

$$g(n) \;=\; \begin{cases} n & \text{si } n \text{ es impar,} \\ n^3 & \text{si } n \text{ es par.} \end{cases}$$

El cociente vale n en los impares y $1/n$ en los pares. Fijado cualquier N, la cola contiene impares arbitrariamente grandes, de modo que no está acotada superiormente y $s_N = \infty$; y contiene pares con $1/n$ tan pequeño como se quiera, todos positivos, de modo que $i_N = 0$. Por lo tanto

$$\limsup_{n\to\infty} \frac{f(n)}{g(n)} = \infty, \qquad \liminf_{n\to\infty} \frac{f(n)}{g(n)} = 0,$$

y aplicando los criterios se concluye que $f \notin \mathcal{O}(g)$ y $f \notin \Omega(g)$. Las dos funciones son incomparables.

El punto que conviene retener del cálculo anterior es que quitar una cantidad finita de términos iniciales no cambia nada, porque siempre sobran infinitos impares e infinitos pares. Esa es la misma idea que expresa el umbral n_0 de la definición original: n_0 concede el derecho a ignorar el principio de la sucesión, y aquí se ve que ejercer ese derecho no ayuda.

Señal de alarma

Una función definida por casos, típicamente por paridad o por rangos, casi siempre está construida para romper la monotonía. El reflejo correcto ante ella es evaluar cada rama por separado antes de intentar probar nada, porque el desenlace habitual es la incomparabilidad.

5. Recurrencias

10 de septiembre de 2026

Resumen rápido. El tema que quedó abierto al final de la clase pasada. Qué es una recurrencia y de dónde sale, el caso de quick sort que explica por qué el mismo algoritmo aparece con dos complejidades distintas en la tabla, y los cuatro métodos para resolverlas: sustitución, iteración, árboles de recursión y el método maestro.

De dónde salen

Las recurrencias se dan cuando un algoritmo se llama a sí mismo. El trabajo que hace sobre una entrada de tamaño n se parte en dos piezas: lo que cuesta acomodar el problema en este nivel, y lo que cuestan las llamadas sobre entradas más chicas. Esa segunda parte no se puede contar directamente, porque es la misma función otra vez. Entonces el tiempo de ejecución no queda como una fórmula cerrada, queda definido en términos de sí mismo.

Definición 5.1. Una recurrencia es una ecuación o desigualdad que describe una función en términos de su valor para entradas más pequeñas.

Ojito...

Una recurrencia sin caso base no define nada. La ecuación sola se desdobla para siempre; es el caso base el que detiene el descenso y le da un valor a todo lo demás. Cuando se omite, como se hace seguido, se sobreentiende que $T(n)$ es constante para n suficientemente chico.

Ejemplo: quick sort

Quick sort escoge un pivote, particiona el arreglo alrededor de él y se llama a sí mismo sobre cada lado. La partición recorre el arreglo una vez, así que cuesta $\mathcal{O}(n)$. Lo que cambia todo es cómo queda el corte, y eso depende del pivote.

Si el pivote parte el arreglo por la mitad, cada llamada genera dos subproblemas de tamaño $n/2$:

$$T(n) = 2\,T\left(\frac{n}{2}\right) + \mathcal{O}(n)\,.$$

Si el pivote resulta ser el mínimo o el máximo, un lado se queda vacío y el otro con todo menos el pivote:

$$T(n) = T(n-1) + \mathcal{O}(n)\,.$$

Idea clave

Son dos recurrencias del mismo algoritmo, y resuelven a $\mathcal{O}(n \log n)$ y $\mathcal{O}(n^2)$ respectivamente. Ahí está la explicación del renglón de quick sort en la tabla 1: la diferencia entre el caso promedio y el peor caso no está en el código, está en qué tan parejo queda el corte.

Los cuatro métodos, de un vistazo

Resolver una recurrencia es encontrarle una forma cerrada, o al menos una cota asintótica. Se nombraron cuatro caminos, que se desarrollan en la clase siguiente.

Método	**En qué consiste**
Sustitución	Adivinar una cota y probarla por inducción matemática
Iteración	Expandir la recurrencia hasta expresarla como una sumatoria en términos de n y las condiciones iniciales
Árbol	Convertir la recurrencia en un árbol cuyos nodos son los costos de los distintos niveles de la recursión
Maestro	Receta para las recurrencias de la forma $T(n) = aT(n/b) + f(n)$. Requiere memorizar tres casos

6. Métodos para resolver recurrencias

15 de septiembre de 2026

Resumen rápido. Los cuatro métodos de la clase pasada, ahora cada uno con un ejemplo resuelto paso a paso. Sustitución con su prueba por inducción fuerte y el problema de las condiciones de frontera, iteración desenrollando hasta una serie geométrica, árboles de recursión sumando por niveles, y el teorema maestro con sus tres casos y el hueco que dejan.

Método de sustitución

Consta de dos pasos, en este orden:

1. **Adivinar** la forma de la solución.
2. **Sustituir** en la recursión donde n decrece, asumir de manera inductiva que la cota es cierta ahí, y probarla para el siguiente valor de n.

Es un método poderoso, pero solo se aplica cuando es fácil adivinar la forma de la respuesta. Sirve tanto para cotas superiores como para inferiores.

Ejemplo resuelto

Ejemplo 6.1. Resolver $T(n) = 2\,T(\lfloor n/2 \rfloor) + n$.

Paso 1. Adivinamos $T(n) = \mathcal{O}(n \lg n)$. Escrito como lo va a usar la inducción, por probar que existe $c > 0$ con

$$T(n) \leq c\,n \lg n.$$

Paso 2. Supongamos que la cota vale para $\lfloor n/2 \rfloor$, o sea

$$T(\lfloor n/2 \rfloor) \leq c\lfloor n/2 \rfloor \lg(\lfloor n/2 \rfloor).$$

Sustituyendo en la recurrencia:

$$\begin{aligned} T(n) &\leq 2\,(c\lfloor n/2 \rfloor \lg(\lfloor n/2 \rfloor)) + n \\ &\leq c\,n \lg(n/2) + n && \text{quitando el piso, que solo achica} \\ &= c\,n \lg n - c\,n \lg 2 + n && \lg(n/2) = \lg n - \lg 2 \\ &= c\,n \lg n - c\,n + n && \lg 2 = 1 \\ &\leq c\,n \lg n. \end{aligned}$$

El último paso pide que $-c\,n + n \leq 0$, es decir $\boxed{c \geq 1}$.

Idea clave

Fíjate en de dónde sale todo el trabajo: el $-c\,n$ aparece solo porque $\lg 2 = 1$, y es justo lo que se come el $+n$ que la recurrencia agrega en cada nivel. Si el término suelto fuera n^2 en vez de n, ese $-c\,n$ ya no alcanzaría y la conjetura $\mathcal{O}(n \lg n)$ se caería.

Las condiciones de frontera

La inducción no está completa hasta probar que la cota vale en la frontera. Y ahí aparece un detalle que tumba la prueba si se pasa por alto.

Ojito...

Con $T(1) = 1$, la cota exige $T(1) \leq c \cdot 1 \cdot \lg 1 = 0$, o sea $1 \leq 0$, que es falso **para cualquier** c. La recurrencia no funciona para $T(1)$.

La salida es que la notación asintótica solo exige que la desigualdad valga de un n_0 en adelante, así que se escoge $n_0 = 2$ y se verifican $n = 2$ y $n = 3$ como casos base:

Valor	De la recurrencia	Lo que pide la cota
$T(2)$	$2T(1) + 2 = 4$	$c\,2\lg 2 = 2c \geq 4 \;\Rightarrow\; c \geq 2$
$T(3)$	$2T(1) + 3 = 5$	$c\,3\lg 3 = 4.755c \geq 5$

Con $c \geq 2$ los dos se cumplen, y como el paso inductivo solo pedía $c \geq 1$, esa misma constante sirve para toda la prueba.

Ojito...

La constante c tiene que ser **una sola** para toda la prueba. No se vale escoger una para el paso inductivo y otra para cada caso base: se elige la más exigente de todas y se usa en todos lados.

Por qué la inducción tiene que ser fuerte

Hay una razón de fondo para que aquí no sirva la inducción ordinaria. La recurrencia manda de $n+1$ a $\lfloor (n+1)/2 \rfloor$, que no es n sino un valor mucho más chico. Suponer la cota solo para n no sirve de nada.

Definición 6.2. En la **inducción fuerte** la hipótesis no es que la propiedad vale para n, sino que vale para *todos* los valores desde el caso base hasta n. Con eso se puede aplicar a cualquier tamaño menor que aparezca.

Ejemplo 6.3. La misma recurrencia, escrita con la hipótesis fuerte.

Hipótesis. Para todo k con $2 \leq k \leq n$,

$$T(k) \leq c\,k \log_2 k.$$

Por demostrar. $T(n+1) \leq c\,(n+1)\log_2(n+1)$.

Paso inductivo. Sea $m = \lfloor (n+1)/2 \rfloor$. Para $n + 1 \geq 4$ se cumple $2 \leq m \leq n$, así que la hipótesis aplica a $T(m)$:

$$\begin{aligned}
T(n+1) &= 2\,T(m) + (n+1) \\
&\leq 2c\,m \log_2 m + (n+1) && \text{por la hipótesis} \\
&\leq c\,(n+1)\log_2\!\left(\tfrac{n+1}{2}\right) + (n+1) && \text{porque } 2m \leq n+1 \\
&= c\,(n+1)\,[\log_2(n+1) - 1] + (n+1) \\
&= c\,(n+1)\log_2(n+1) + (n+1)(1-c).
\end{aligned}$$

Si $c \geq 1$ entonces $(n+1)(1-c) \leq 0$, y por lo tanto

$$T(n+1) \leq c\,(n+1)\log_2(n+1).$$

Idea clave

Los casos base $n = 2$ y $n = 3$ no son un capricho. Se necesita $m \geq 2$ para poder aplicar la hipótesis, y $m = 1$ ocurre justo cuando $n + 1 = 3$. Cubrir 2 y 3 a mano deja el paso general trabajando siempre con m dentro del rango.

Cómo adivinar la cota

Es la debilidad del método: **no hay una manera general** de adivinar la solución correcta. Requiere experiencia y creatividad. Dos heurísticas que sí ayudan:

- Dibujar un árbol de recursión para generar un buen valor inicial, y luego probarlo con sustitución.
- Si la recurrencia se parece a otra ya resuelta, probar con una solución parecida.

Método de iteración

No requiere adivinar la respuesta, y esa es toda su gracia. Se expande la recurrencia dentro de sí misma una y otra vez hasta expresarla como una sumatoria de términos que dependen solo de n y de las condiciones iniciales. Después se acota esa sumatoria.

El costo es el álgebra: puede ponerse pesado rápido. A veces se itera solo lo suficiente para intuir la forma de la solución y de ahí se sigue con sustitución.

Ejemplo 6.4. Resolver $T(n) = 3\,T(\lfloor n/4 \rfloor) + n$.

Paso 1. Desenrollar.

$$\begin{aligned} T(n) &= n + 3\,T(\lfloor n/4 \rfloor) \\ &= n + 3\,(\lfloor n/4 \rfloor + 3\,T(\lfloor n/16 \rfloor)) \\ &= n + 3\,(\lfloor n/4 \rfloor + 3\,(\lfloor n/16 \rfloor + 3\,T(\lfloor n/64 \rfloor))) \\ &= n + 3\lfloor n/4 \rfloor + 9\lfloor n/16 \rfloor + 27\,T(\lfloor n/64 \rfloor). \end{aligned}$$

Paso 2. Leer el patrón. El término i-ésimo de la serie es $3^i \lfloor n/4^i \rfloor$.

Paso 3. Saber dónde parar. La iteración llega al caso base cuando $\lfloor n/4^i \rfloor = 1$, o sea cuando i alcanza $\log_4 n$.

Paso 4. Acotar la suma. Usando $\lfloor n/4^i \rfloor \leq n/4^i$:

$$\begin{aligned} T(n) &\leq n + \tfrac{3n}{4} + \tfrac{9n}{16} + \tfrac{27n}{64} + \cdots + 3^{\log_4 n}\,\Theta(1) \\ &\leq n \sum_{i=0}^{\infty} \left(\tfrac{3}{4}\right)^i + \Theta\left(n^{\log_4 3}\right) \\ &= 4n + o(n) \\ &= \mathcal{O}(n)\,. \end{aligned}$$

Idea clave

Los dos pasos que hacen el truco. Primero, la serie $\sum (3/4)^i$ es geométrica decreciente, así que converge a $1/(1 - 3/4) = 4$ aunque se extienda al infinito: acotar de más no cuesta nada y ahorra manejar el límite exacto. Segundo, $n^{\log_4 3} \approx n^{0.79}$ crece más lento que n, así que es $o(n)$ y desaparece dentro del $\mathcal{O}(n)$.

Árboles de recursión

Es la iteración, pero dibujada. Cada nodo representa el costo de un subproblema dentro del conjunto de llamadas recursivas. Se suman los costos por nivel y luego se suman los niveles. Sirve sobre todo para **generar una buena conjetura**, que después se comprueba con sustitución.

Ejemplo 6.5. Resolver $T(n) = 3\,T(\lfloor n/4 \rfloor) + \Theta(n^2)$. Se trabaja con $T(n) = 3\,T(n/4) + c\,n^2$, con $c > 0$, y se supone que n es potencia exacta de 4 para no cargar con los pisos.

Paso 1. Cuántos niveles hay. El subproblema a profundidad i tiene tamaño $n/4^i$. Llega a 1 cuando $n/4^i = 1$, o sea $i = \log_4 n$. El árbol tiene entonces $\log_4 n + 1$ niveles, contando el 0.

Paso 2. Cuánto cuesta cada nivel. Cada nodo se abre en tres, así que a profundidad i hay 3^i nodos, y cada uno cuesta $c\,(n/4^i)^2$. Multiplicando:

$$3^i \cdot c\left(\frac{n}{4^i}\right)^2 = \frac{3^i\,c\,n^2}{16^i} = \left(\frac{3}{16}\right)^i c\,n^2.$$

Nivel	Nodos	Tamaño	Costo del nivel
0	1	n	$c\,n^2$
1	3	$n/4$	$(3/16)\,c\,n^2$
2	9	$n/16$	$(3/16)^2 c\,n^2$
i	3^i	$n/4^i$	$(3/16)^i c\,n^2$

Paso 3. Las hojas. El último nivel, a profundidad $\log_4 n$, tiene $3^{\log_4 n} = n^{\log_4 3}$ hojas, cada una con costo $T(1)$, para un total de $\Theta\left(n^{\log_4 3}\right)$.

Paso 4. Sumar todo.

$$\begin{aligned} T(n) &= \sum_{i=0}^{\log_4 n - 1} \left(\frac{3}{16}\right)^i c\,n^2 + \Theta\left(n^{\log_4 3}\right) \\ &< \sum_{i=0}^{\infty} \left(\frac{3}{16}\right)^i c\,n^2 + \Theta\left(n^{\log_4 3}\right) \\ &= \frac{1}{1 - 3/16}\,c\,n^2 + \Theta\left(n^{\log_4 3}\right) \\ &= \frac{16}{13}\,c\,n^2 + \Theta\left(n^{\log_4 3}\right) = \mathcal{O}\left(n^2\right). \end{aligned}$$

Y como la raíz sola ya cuesta $c\,n^2$, la cota inferior también es n^2, así que $T(n) = \Theta(n^2)$.

La identidad del exponente

En el paso 3 se usó $3^{\log_4 n} = n^{\log_4 3}$, que parece un truco y no lo es. Es un caso de $a^{\log_c b} = b^{\log_c a}$, y sale de escribir los dos lados como exponenciales:

$$3^{\log_4 n} = 3^{\frac{\ln n}{\ln 4}} = \left(e^{\ln 3}\right)^{\frac{\ln n}{\ln 4}} = e^{\frac{(\ln 3)(\ln n)}{\ln 4}}, \qquad n^{\log_4 3} = e^{\frac{(\ln n)(\ln 3)}{\ln 4}}.$$

Los exponentes son iguales porque la multiplicación es conmutativa.

La serie geométrica finita

La otra pieza que se usó. Para $S = 1 + r + r^2 + \cdots + r^{k-1}$:

$$\begin{aligned} S &= 1 + r + r^2 + \cdots + r^{k-1} \\ r\,S &= \quad r + r^2 + \cdots + r^{k-1} + r^k. \end{aligned}$$

Restando la segunda de la primera se cancelan todos los términos de en medio y queda $S - r\,S = 1 - r^k$, de donde

$$S = \frac{1 - r^k}{1 - r}.$$

Cuando $|r| < 1$ y la suma es infinita, $r^k \to 0$ y se llega a $\sum_{k=0}^{\infty} r^k = 1/(1-r)$, que es la forma que se usó arriba.

Idea clave

Que $r = 3/16 < 1$ es lo que decide el resultado: el costo baja geométricamente conforme se desciende, así que **manda la raíz**. Si r fuera mayor que 1 mandarían las hojas, y si fuera exactamente 1 todos los niveles costarían lo mismo y habría que multiplicar por el número de niveles. Esos tres escenarios son exactamente los tres casos del método maestro.

Método maestro

Es una receta, sin inducción ni desenrollado. Aplica a las recurrencias de la forma

$$T(n) = a\,T(n/b) + f(n), \qquad a \geq 1, \quad b > 1,$$

con a y b constantes y $f(n)$ asintóticamente positiva. Describe un algoritmo que parte un problema de tamaño n en a subproblemas de tamaño n/b, los resuelve recursivamente en tiempo $T(n/b)$, y paga $f(n)$ por dividir y combinar. Da igual si n/b se lee como $\lfloor n/b \rfloor$ o como $\lceil n/b \rceil$.

Teorema 6.6 (Teorema maestro)**.** *Con $T(n) = a\,T(n/b) + f(n)$:*

1. *Si $f(n) = \mathcal{O}\left(n^{\log_b a - \epsilon}\right)$ para alguna constante $\epsilon > 0$, entonces $T(n) = \Theta\left(n^{\log_b a}\right)$.*
2. *Si $f(n) = \Theta\left(n^{\log_b a}\right)$, entonces $T(n) = \Theta\left(n^{\log_b a} \lg n\right)$.*
3. *Si $f(n) = \Omega\left(n^{\log_b a + \epsilon}\right)$ para alguna constante $\epsilon > 0$, y además $a\,f(n/b) \leq c\,f(n)$ para alguna constante $c < 1$ y todo n suficientemente grande, entonces $T(n) = \Theta(f(n))$.*

Idea clave

Los tres casos son la misma pregunta: comparar $f(n)$, que es lo que cuesta la raíz, contra $n^{\log_b a}$, que es lo que cuestan las hojas. **Gana la mayor de las dos.** Si empatan, todos los niveles cuestan lo mismo y se multiplica por el factor logarítmico, que no es otra cosa que el número de niveles.

La letra chica

Ojito...

En los casos 1 y 3 no basta con que una función sea mayor que la otra: tiene que serlo **polinómicamente**, o sea por un factor de n^ϵ con $\epsilon > 0$. Y el caso 3 pide además la condición de regularidad $a\,f(n/b) \leq c\,f(n)$, que se cumple para casi cualquier polinomio pero hay que verificarla.
La consecuencia es que **los tres casos no cubren todas las posibilidades**. Hay recurrencias que caen en los huecos y para las que el teorema simplemente no dice nada.

Los cuatro ejemplos

Ejemplo 6.7 (Caso 1)**.** $T(n) = 9\,T(n/3) + n$. Aquí $a = 9$, $b = 3$, $f(n) = n$, y $n^{\log_b a} = n^{\log_3 9} = n^2$. Como $f(n) = n = \mathcal{O}\big(n^{2-\epsilon}\big)$ con $\epsilon = 1$, las hojas dominan:

$$T(n) = \Theta\Big(n^2\Big)\,.$$

Ejemplo 6.8 (Caso 2)**.** $T(n) = T(2n/3) + 1$. Aquí $a = 1$, $b = 3/2$, $f(n) = 1$, y $n^{\log_{3/2} 1} = n^0 = 1$. Como $f(n) = \Theta(1)$ empata con $n^{\log_b a}$, todos los niveles cuestan lo mismo:

$$T(n) = \Theta(\lg n)\,.$$

Ejemplo 6.9 (Caso 3)**.** $T(n) = 3\,T(n/4) + n \lg n$. Aquí $a = 3$, $b = 4$, $f(n) = n \lg n$, y $n^{\log_4 3} = \mathcal{O}(n^{0.793})$. Como $f(n) = \Omega\Big(n^{\log_4 3 + \epsilon}\Big)$ con $\epsilon \approx 0.2$, falta verificar la regularidad:

$$a\,f(n/b) = 3\,\frac{n}{4}\lg\frac{n}{4} \leq \frac{3}{4}\,n \lg n = c\,f(n) \quad \text{con } c = 3/4 < 1.$$

Se cumple, así que manda la raíz:

$$T(n) = \Theta(n \lg n)\,.$$

Ejemplo 6.10 (Ninguno de los tres)**.** $T(n) = 2\,T(n/2) + n \lg n$. Tiene la forma correcta, con $a = 2$, $b = 2$, $f(n) = n \lg n$ y $n^{\log_2 2} = n$.

Parecería el caso 3, porque $n \lg n$ sí es asintóticamente mayor que n. El problema es que **no lo es polinómicamente**: el cociente

$$\frac{f(n)}{n^{\log_b a}} = \frac{n \lg n}{n} = \lg n$$

es asintóticamente *menor* que n^ϵ para cualquier $\epsilon > 0$ positivo. La recurrencia cae en el hueco entre el caso 2 y el caso 3, y el teorema maestro no la resuelve.

Ojito...

Ese último ejemplo es el que hay que tener presente: que una recurrencia tenga la forma $a\,T(n/b) + f(n)$ **no garantiza** que el método maestro aplique. Siempre hay que comprobar cuál de los tres casos se cumple, y aceptar que a veces ninguno lo hace. Ahí toca volver a los árboles de recursión y a la sustitución.

Cuál usar

Método	Qué pide	Qué entrega
Sustitución	Una conjetura previa	Prueba formal
Iteración	Aguante algebraico	La forma cerrada
Árbol	Dibujar y sumar niveles	Una conjetura
Maestro	Que encaje en un caso	La cota, directo

Idea clave

En la práctica se usan en pareja. El árbol da la conjetura, la sustitución la prueba. El maestro se intenta primero porque si aplica ahorra todo el trabajo, pero hay que verificar el caso, no suponerlo.

7. Heaps

17 de septiembre de 2026

Resumen rápido. Se cerró el método maestro y arrancó ordenamiento. El heap: qué es, cómo se guarda un árbol binario en un arreglo plano, la propiedad que lo define y su altura. Luego las operaciones, con Heapify como la pieza central, y la construcción del heap, que resulta costar $\mathcal{O}(n)$ y no $\mathcal{O}(n \lg n)$ como parecería.

Qué es un heap

Definición 7.1. Un **heap** es una estructura de datos binaria: un arreglo que representa un árbol binario **completo**, o sea lleno en todos sus niveles salvo tal vez el último, que se llena de izquierda a derecha.

Cada nodo del árbol es un elemento del arreglo y guarda su valor. El arreglo lleva dos atributos:

$\mathit{length}[A]$	Número de elementos del arreglo A
$\mathit{heap\text{-}size}[A]$	Número de elementos que de verdad forman parte del heap

Siempre se cumple $\mathit{heap\text{-}size}[A] \leq \mathit{length}[A]$, y la raíz del árbol es $A[1]$.

Ojito...

Los dos atributos no son lo mismo y la diferencia importa. El arreglo puede ser más grande que el heap que contiene, y en heapsort esa diferencia es justo el mecanismo: el heap se va encogiendo dentro del mismo arreglo mientras la parte de atrás guarda lo ya ordenado.

Navegar el árbol con aritmética

Lo que hace útil al heap es que no necesita apuntadores. Dado el índice i de un nodo:

Padre(i)	**devolver** $\lfloor i/2 \rfloor$
Izq(i)	**devolver** $2i$
Der(i)	**devolver** $2i + 1$

Idea clave

Estas tres fórmulas funcionan **porque** el árbol es completo. Al recorrer por niveles, el nivel k ocupa las posiciones 2^k a $2^{k+1} - 1$, así que los hijos de i caen siempre en $2i$ y $2i + 1$ sin dejar huecos. Si el árbol tuviera agujeros, la cuenta se rompería y habría que volver a los apuntadores.

Ejemplo 7.2. Sobre el arreglo $A = [16, 14, 10, 8, 7, 9, 3, 2, 4, 1]$ con $\mathit{heap\text{-}size} = 10$:

i	$A[i]$	**Padre**	**Izq**	**Der**
1	16	–	$A[2] = 14$	$A[3] = 10$
2	14	$A[1] = 16$	$A[4] = 8$	$A[5] = 7$
5	7	$A[2] = 14$	$A[10] = 1$	no existe
8	2	$A[4] = 8$	no existe	no existe

El índice 5 solo tiene hijo izquierdo porque $2 \cdot 5 + 1 = 11 > 10$. Los índices del 6 en adelante son hojas.

La propiedad de heap

Definición 7.3. Un **MAX-HEAP** cumple $A[\text{Padre}(i)] \geq A[i]$ para todo nodo i distinto de la raíz. Un **MIN-HEAP** cumple lo contrario, $A[\text{Padre}(i)] \leq A[i]$.

Ojito...

La propiedad solo relaciona a cada nodo con su **padre**. No dice nada entre hermanos ni entre ramas distintas, así que un heap **no es** un árbol binario de búsqueda y recorrerlo en orden no da la secuencia ordenada. Lo único garantizado es que el máximo está en la raíz.

Altura

La **altura de un nodo** es el número de arcos de la ruta simple más larga que baja de ese nodo a una hoja. La **altura del árbol** es la de su raíz.

Como el árbol es completo, cada nivel duplica al anterior, así que con x niveles caben

$$n = \underbrace{2 \cdot 2 \cdots 2}_{x} = 2^x \quad \Longrightarrow \quad x = \log_2 n.$$

Por lo tanto la altura del árbol es $\Theta(\lg n)$, y ese logaritmo es el que va a aparecer en el costo de casi todas las operaciones.

Operaciones sobre un MAX-HEAP

Máximo

Está en la raíz por definición, así que se lee $A[1]$ y ya: $\mathcal{O}(1)$.

Inserción

Primero hay que **respetar la forma** del heap, o sea que siga siendo un árbol completo. Eso obliga a meter el elemento nuevo en la única posición que no abre huecos: al final del arreglo.

1. Insertar al final del arreglo: $\mathcal{O}(1)$.
2. Verificar si se mantiene la propiedad de heap: $\mathcal{O}(1)$, es una sola comparación contra el padre.
3. Si no se mantiene, intercambiar con el padre y repetir hacia arriba, hasta la raíz si hace falta.

El paso 3 sube a lo más la altura del árbol, así que la inserción cuesta $\mathcal{O}(\lg n)$.

Eliminación de la raíz

Eliminar el último nodo es $\mathcal{O}(1)$. Eliminar la raíz, que es lo que de verdad se necesita, se resuelve con un truco:

1. Intercambiar el último nodo con la raíz: $\mathcal{O}(1)$.
2. Eliminar el último nodo, que ahora contiene el valor que queríamos sacar: $\mathcal{O}(1)$.
3. La raíz quedó con un valor que probablemente viola la propiedad. Hay que restaurarla, y para eso está Heapify.

Heapify

Es la operación central. El escenario es muy específico:

- **Entrada:** un arreglo A y un índice i.
- **Supuesto:** los subárboles con raíz en Izq(i) y Der(i) *ya son heaps* válidos.
- **Problema:** $A[i]$ puede ser menor que alguno de sus hijos, y eso viola la propiedad de MAX-HEAP.

La solución es buscar el mayor de los dos hijos e intercambiarlo con el padre. Pero el hijo que recibió el valor viejo puede ahora violar la propiedad en su propio subárbol, así que se repite el procedimiento ahí, con ese hijo como raíz.

Algoritmo 7.1 Max-Heapify(A, i)

```
1:  l ← Izq(i)
2:  r ← Der(i)
3:  si l ≤ heap-size[A] y A[l] > A[i] entonces
4:      mayor ← l
5:  si no
6:      mayor ← i
7:  fin si
8:  si r ≤ heap-size[A] y A[r] > A[mayor] entonces
9:      mayor ← r
10: fin si
11: si mayor ≠ i entonces
12:     intercambiar A[i] ↔ A[mayor]
13:     Max-Heapify(A, mayor)
14: fin si
```

Ejemplo 7.4. Sobre $A = [16, 4, 10, 14, 7, 9, 3, 2, 8, 1]$ con *heap-size* $= 10$, llamando Max-Heapify$(A, 2)$. El nodo $A[2] = 4$ es menor que sus hijos, así que viola la propiedad.

Paso	i	Hijos	Acción
1	2	$A[4] = 14$, $A[5] = 7$	mayor es 14, intercambiar con $A[2]$
2	4	$A[8] = 2$, $A[9] = 8$	mayor es 8, intercambiar con $A[4]$
3	9	sin hijos	es hoja, se detiene

El arreglo queda $A = [16, 14, 10, 8, 7, 9, 3, 2, 4, 1]$. El 4 bajó dos niveles, de la posición 2 a la 9, empujando hacia arriba a los mayores.

Costo de Heapify

En un subárbol de tamaño n con raíz en i, el trabajo son dos cosas: comparar $A[i]$ contra sus dos hijos, que es $\Theta(1)$, más la llamada recursiva sobre uno de los hijos. La recurrencia es

$$T(n) = T(n/2) + \Theta(1)\,,$$

que por el método maestro cae en el caso 2 con $a = 1$, $b = 2$, $n^{\log_2 1} = n^0 = 1$, y da

$$T(n) = \mathcal{O}(\lg n)\,.$$

Dicho de otro modo, Heapify sobre un nodo de altura h cuesta $\mathcal{O}(h)$: baja un nivel por llamada y no puede bajar más que la altura.

Observación. La cota fina es $T(n) \leq T(2n/3) + \Theta(1)$, no $T(n/2)$. El peor caso ocurre cuando el último nivel está lleno exactamente a la mitad, y ahí el subárbol izquierdo alcanza $2n/3$ nodos en vez de $n/2$. El resultado no cambia, porque $n^{\log_{3/2} 1} = 1$ igual y el caso 2 sigue dando $\mathcal{O}(\lg n)$, pero conviene saber de dónde sale ese $2/3$ cuando aparezca en el libro.

Construcción del heap

La forma obvia es insertar los elementos uno por uno: meter al final, verificar la propiedad, subir si hace falta. Son n inserciones de $\mathcal{O}(\lg n)$ cada una, o sea $\mathcal{O}(n \lg n)$.

Se puede mejor. La idea es aplicar Heapify de abajo hacia arriba sobre la mitad de los nodos, apoyándose en dos observaciones:

- Los elementos $A[\lfloor n/2 \rfloor + 1 \ldots n]$ son **hojas** del árbol, y una hoja ya es un heap válido de un solo nodo. No hay nada que hacer con ellas.
- Si se recorren los nodos **en orden decreciente** de índice, cuando le toca a i los subárboles de sus hijos ya fueron procesados y son heaps. Eso es exactamente el supuesto que Heapify necesita.

Algoritmo 7.2 Build-Max-Heap(A)

```
1: heap-size[A] ← length[A]
2: para i ← ⌊length[A]/2⌋ bajando hasta 1 hacer
3:     Max-Heapify(A, i)
4: fin para
```

Ejemplo 7.5. Construir el heap sobre $A = [4, 1, 3, 2, 16, 9, 10, 14, 8, 7]$, con $n = 10$. El ciclo va de $i = \lfloor 10/2 \rfloor = 5$ hasta 1.

i	**Qué pasa**	**Arreglo después**
5	$A[5] = 16$ contra su hijo $A[10] = 7$. Ya cumple, no se mueve nada.	4 1 3 2 16 9 10 14 8 7
4	$A[4] = 2$ contra 14 y 8. Sube el 14.	4 1 3 14 16 9 10 2 8 7
3	$A[3] = 3$ contra 9 y 10. Sube el 10.	4 1 10 14 16 9 3 2 8 7
2	$A[2] = 1$ contra 14 y 16. Sube el 16; el 1 sigue bajando y cambia con el 7.	4 16 10 14 7 9 3 2 8 1
1	$A[1] = 4$ contra 16 y 10. Sube el 16; el 4 baja hasta la posición 9.	16 14 10 8 7 9 3 2 4 1

Fíjate en los renglones de $i = 2$ y $i = 1$: ahí Heapify no hizo un solo intercambio sino una cadena de ellos, porque el valor desplazado siguió violando la propiedad más abajo.

Por qué cuesta $\mathcal{O}(n)$ y no $\mathcal{O}(n \lg n)$

La cuenta rápida dice: hay $\mathcal{O}(n)$ llamadas a Heapify y cada una cuesta $\mathcal{O}(\lg n)$, así que $\mathcal{O}(n \lg n)$. Esa cota es **correcta pero no justa**, y el motivo es que casi todas las llamadas son sobre nodos bajitos, donde Heapify apenas trabaja.

La cota fina sale de contar por altura, apoyándose en una propiedad del árbol completo: en un heap de n elementos hay a lo más $\lceil n/2^{h+1} \rceil$ nodos de altura h. Como Heapify en un nodo de altura h cuesta $\mathcal{O}(h)$, el total es

$$\sum_{h=0}^{\lfloor \lg n \rfloor} \left\lceil \frac{n}{2^{h+1}} \right\rceil \mathcal{O}(h) = \mathcal{O}\left(n \cdot \frac{1}{2} \sum_{h=0}^{\lfloor \lg n \rfloor} \frac{h}{2^h} \right).$$

Falta evaluar esa sumatoria, y el truco es derivar la serie geométrica. Partiendo de $\sum_{k=0}^{\infty} x^k = 1/(1-x)$ para $|x| < 1$, se derivan los dos lados:

$$\frac{d}{dx}\left(\sum_{k=0}^{\infty} x^k \right) = \frac{d}{dx}\left(\frac{1}{1-x} \right) \quad \Longrightarrow \quad \sum_{k=0}^{\infty} k\, x^{k-1} = \frac{1}{(1-x)^2}.$$

Multiplicando ambos lados por x:

$$\sum_{k=0}^{\infty} k\, x^k = \frac{x}{(1-x)^2}.$$

En nuestro caso $k = h$ y $x = 1/2$, así que

$$\sum_{h=0}^{\infty} \frac{h}{2^h} = \frac{1/2}{(1-1/2)^2} = \frac{1/2}{1/4} = 2.$$

Sustituyendo, el costo de Build-Max-Heap es

$$\mathcal{O}\Big(n \cdot \tfrac{1}{2} \cdot 2 \Big) = \mathcal{O}(n)\,.$$

Idea clave

De dónde sale la ganancia: la mitad de los nodos son hojas y cuestan cero, un cuarto tiene altura 1 y cuesta uno, un octavo tiene altura 2, y así. Los nodos caros son poquísimos, y la serie $\sum h/2^h$ converge a 2 en vez de crecer con $\lg n$. Por eso construir el heap de golpe es $\mathcal{O}(n)$, mientras que construirlo insertando uno por uno es $\mathcal{O}(n \lg n)$.

Resumen

Operación	**Costo**
Encontrar el máximo	$\mathcal{O}(1)$
Insertar	$\mathcal{O}(\lg n)$
Eliminar la raíz, vía Heapify	$\mathcal{O}(\lg n)$
Heapify en un nodo de altura h	$\mathcal{O}(h)$
Construir insertando uno por uno	$\mathcal{O}(n \lg n)$
Build-Max-Heap	$\mathcal{O}(n)$

8. Mochila fraccional, actividades y divide y vencerás *8 de octubre de 2026*

Resumen rápido. Dos problemas clásicos que se resuelven con algoritmos voraces: la mochila fraccional y la selección de actividades cuyos horarios se traslapan. Para cada uno, el criterio voraz que funciona, un ejemplo completo, por qué da el óptimo y qué criterios parecidos fallan. Al final arrancó la técnica de divide y vencerás.

Repaso: algoritmos voraces

Un algoritmo voraz (*greedy*) resuelve un problema de optimización tomando, en cada paso, la opción que se ve mejor en ese momento, sin revisar nunca una decisión ya tomada. Suelen ser lineales o polinomiales. El riesgo es que lo mejor en el momento no siempre lleva al óptimo global, así que cada algoritmo voraz necesita un argumento de por qué su criterio funciona en ese problema en particular. Los dos de esta clase son el ejemplo de cuándo sí, y una variante de la mochila es el ejemplo de cuándo no.

Mochila fraccional

Definición 8.1. Hay n objetos; el objeto i pesa w_i y vale v_i. La mochila aguanta un peso máximo W. De cada objeto se puede tomar una fracción $x_i \in [0,1]$, como si fuera polvo de oro y no un lingote. Se busca

$$\max \sum_{i=1}^{n} v_i\, x_i \qquad \text{sujeto a} \qquad \sum_{i=1}^{n} w_i\, x_i \le W.$$

El criterio voraz

Lo que importa no es el valor de un objeto sino cuánto vale cada unidad de peso que ocupa: su **densidad** v_i/w_i. El algoritmo ordena los objetos de mayor a menor densidad y los va metiendo completos mientras quepan. Cuando el siguiente ya no cabe completo, mete la fracción que alcance a llenar la mochila y termina.

Algoritmo 8.1 Mochila-Fraccional(w, v, W)

```
ordenar los objetos de mayor a menor v_i/w_i
libre ← W,    valor ← 0
para i ← 1 hasta n hacer
    si w_i ≤ libre entonces
        x_i ← 1
    si no
        x_i ← libre/w_i
    fin si
    libre ← libre − x_i w_i
    valor ← valor + x_i v_i
    si libre = 0 entonces
        terminar
    fin si
fin para
devolver valor
```

Ordenar cuesta $\mathcal{O}(n \lg n)$ y la pasada es $\mathcal{O}(n)$, así que el algoritmo es $\mathcal{O}(n \lg n)$.

Ejemplo 8.2. Mochila de $W = 50$ con tres objetos:

Objeto	Peso	Valor	Densidad
1	10	60	6
2	20	100	5
3	30	120	4

Ya están en orden de densidad. Paso a paso:

Objeto	Qué se toma	Libre después	Valor acumulado
1	completo, 10 de peso	40	60
2	completo, 20 de peso	20	160
3	no cabe completo: $20/30 = 2/3$	0	$160 + \frac{2}{3} \cdot 120 = 240$

El valor máximo es 240.

Por qué da el óptimo

Supóngase una solución que deja espacio sin usar de un objeto denso mientras carga un poco de uno menos denso. Si se saca una unidad de peso del menos denso y se mete una del más denso, el peso no cambia y el valor sube, o se queda igual si las densidades empatan. Repitiendo ese intercambio, cualquier solución se puede convertir en la del algoritmo sin perder valor. Por eso ninguna solución vale más que la voraz.

Ojito...

Lo que hace funcionar al criterio es poder partir los objetos. En la **mochila 0/1**, donde cada objeto se toma completo o no se toma, el mismo criterio falla. Con el ejemplo de arriba, el voraz mete los objetos 1 y 2, y el 3 ya no cabe: vale 160. Pero los objetos 2 y 3 juntos pesan exactamente 50 y valen 220. Al no poder partir el objeto 3, el espacio que deja libre el voraz se desperdicia. La mochila 0/1 se resuelve con programación dinámica, no con un algoritmo voraz.

Selección de actividades

Es el problema de los horarios que se traslapan.

Definición 8.3. Hay n actividades que quieren usar el mismo recurso, por ejemplo un salón, y solo una puede usarlo a la vez. La actividad a_i ocupa el intervalo $[s_i, f_i)$: empieza en s_i y termina en f_i. Dos actividades son **compatibles** si sus intervalos no se traslapan, o sea si una termina antes de que empiece la otra, $f_i \leq s_j$ o $f_j \leq s_i$. Se busca el mayor número posible de actividades compatibles entre sí.

El criterio voraz

Elegir siempre la actividad que **termina primero** entre las que todavía son compatibles con lo ya elegido. La razón: terminar pronto deja libre el mayor tiempo posible para todas las demás.

Algoritmo 8.2 Seleccion-Actividades(s, f)

```
ordenar las actividades por tiempo de fin f_i creciente
A ← {a_1},   ultima ← 1
para i ← 2 hasta n hacer
    si s_i ≥ f_ultima entonces
        A ← A ∪ {a_i}
        ultima ← i
    fin si
fin para
devolver A
```

Igual que la mochila: $\mathcal{O}(n \lg n)$ por el ordenamiento y $\mathcal{O}(n)$ por la pasada.

Ejemplo 8.4. Once actividades, ya ordenadas por su tiempo de fin:

i	1	2	3	4	5	6	7	8	9	10	11
s_i	1	3	0	5	3	5	6	8	8	2	12
f_i	4	5	6	7	9	9	10	11	12	14	16

Se recorren en orden, comparando el inicio de cada una contra el fin de la última elegida:

Actividad	**Intervalo**	**Fin de la última**	**Decisión**
a_1	$[1, 4)$	—	se toma
a_2	$[3, 5)$	4	$3 < 4$, se traslapa
a_3	$[0, 6)$	4	se traslapa
a_4	$[5, 7)$	4	$5 \geq 4$, se toma
a_5	$[3, 9)$	7	se traslapa
a_6	$[5, 9)$	7	se traslapa
a_7	$[6, 10)$	7	se traslapa
a_8	$[8, 11)$	7	$8 \geq 7$, se toma
a_9	$[8, 12)$	11	se traslapa
a_{10}	$[2, 14)$	11	se traslapa
a_{11}	$[12, 16)$	11	$12 \geq 11$, se toma

Resultado: $\{a_1, a_4, a_8, a_{11}\}$, cuatro actividades. Revisando todos los subconjuntos posibles, ninguno compatible tiene cinco.

Por qué da el óptimo

Sea a_1 la actividad que termina primero, y tómese cualquier solución óptima. Si no empieza con a_1, se le quita su primera actividad y se pone a_1 en su lugar. Como a_1 termina antes o al mismo tiempo, no choca con nada de lo que seguía, y la solución sigue teniendo el mismo número de actividades. Así que siempre existe una solución óptima que empieza con a_1. Después de elegirla, lo que queda es el mismo problema con las actividades que empiezan después de f_1, y el argumento se repite.

Criterios que parecen razonables y fallan

- **La que empieza primero.** Con $[0, 10)$, $[1, 2)$ y $[3, 4)$ elige $[0, 10)$, que choca con las otras dos: se queda con una sola. El óptimo es dos, $[1, 2)$ y $[3, 4)$.

- **La más corta.** Con $[0,5)$, $[4,6)$ y $[5,10)$ elige $[4,6)$, que dura 2 pero se traslapa con las otras dos: otra vez una sola. El óptimo es $[0,5)$ y $[5,10)$.

Las dos fallan por lo mismo: miran una propiedad de la actividad que no dice cuánto tiempo deja libre. El tiempo de fin es justo eso.

Observación. Hay un problema hermano que también se resuelve con un voraz: en lugar de elegir cuántas actividades caben en un salón, programarlas *todas* usando el menor número de salones. Ahí se ordenan por tiempo de *inicio* y cada actividad va a un salón que ya esté libre, o se abre uno nuevo si no hay.

Divide y vencerás

La idea es simple: dado un problema grande, se divide en subproblemas parecidos de menor tamaño, se resuelven, y con sus soluciones se arma la del problema general. Formalmente son tres pasos:

1. **Dividir** el problema en a subproblemas del mismo tipo, cada uno de tamaño n/b.
2. **Vencer**: resolver cada subproblema recursivamente. Cuando uno ya es lo bastante chico, el caso base, se resuelve directo.
3. **Combinar** las soluciones de los subproblemas en la solución del problema original.

Si dividir y combinar cuestan $f(n)$, el tiempo total cumple

$$T(n) = a\,T(n/b) + f(n),$$

que es justo la forma que resuelve el método maestro de la clase del 15 de septiembre. Por eso las recurrencias se vieron antes: son la herramienta para medir estos algoritmos.

Ejemplo 8.5 (Ordenamiento por mezcla)**.** Para ordenar $[5,2,4,7,1,3,2,6]$ se parte a la mitad hasta llegar a arreglos de un elemento, que ya están ordenados, y luego se van mezclando de dos en dos:

Etapa	**Arreglos**
dividir	$[5,2,4,7]$ $[1,3,2,6]$
dividir	$[5,2]$ $[4,7]$ $[1,3]$ $[2,6]$
caso base	$[5]$ $[2]$ $[4]$ $[7]$ $[1]$ $[3]$ $[2]$ $[6]$
combinar	$[2,5]$ $[4,7]$ $[1,3]$ $[2,6]$
combinar	$[2,4,5,7]$ $[1,2,3,6]$
combinar	$[1,2,2,3,4,5,6,7]$

Mezclar dos arreglos ya ordenados cuesta $\Theta(n)$: basta comparar sus primeros elementos y sacar el menor, una y otra vez. Son dos subproblemas de la mitad del tamaño:

$$T(n) = 2\,T(n/2) + \Theta(n)\,.$$

Con $a = 2$ y $b = 2$, $n^{\log_2 2} = n$, que es del mismo orden que $f(n)$: caso 2 del método maestro, $T(n) = \Theta(n \lg n)$.

Idea clave

La diferencia con un algoritmo voraz está en cuántos caminos se siguen. El voraz toma una decisión y se queda con un solo subproblema, el que deja esa decisión. Divide y vencerás resuelve *todos* los subproblemas en que partió el original y después combina sus respuestas.

9. Multiplicación de enteros grandes

13 de octubre de 2026

Resumen rápido. Divide y vencerás aplicado a multiplicar dos enteros de n dígitos. El método de la escuela cuesta $\Theta(n^2)$. Partir cada número a la mitad da cuatro multiplicaciones de la mitad del tamaño y el mismo $\Theta(n^2)$: dividir por sí solo no ahorra nada. El truco de Karatsuba obtiene el término de en medio con una sola multiplicación en lugar de dos, y con tres subproblemas en lugar de cuatro el costo baja a $\Theta\left(n^{\log_2 3}\right) \approx \Theta(n^{1.585})$.

El problema

Multiplicar dos enteros x y y de n dígitos cada uno. El procesador multiplica números de 64 bits en una instrucción, pero la criptografía trabaja con números mucho más grandes: una llave RSA de 2048 bits tiene unos 617 dígitos decimales. Las bibliotecas de enteros de tamaño arbitrario, como los `int` de Python, tienen que multiplicar números así con operaciones sobre dígitos. El costo se mide en operaciones de un dígito.

El método de la escuela

Se multiplica x por cada dígito de y, se recorre cada renglón un lugar a la izquierda y se suman.

Ejemplo 9.1 (1234×5678)**.** Los cuatro renglones y su suma:

$$\begin{array}{rr} 1234 \times 8 = & 9872 \\ 1234 \times 7 = & 8638 \\ 1234 \times 6 = & 7404 \\ 1234 \times 5 = & 6170 \\ \hline \text{suma:} & 7006652 \end{array}$$

Cada uno de los 4 dígitos de y se multiplica por cada uno de los 4 de x: 16 productos de un dígito.

Con n dígitos son n^2 productos de un dígito, más sumar n renglones de unos $2n$ dígitos. El método es $\Theta(n^2)$. Duplicar el tamaño de los números cuadruplica el tiempo.

Primer intento: partir a la mitad

Con $m = n/2$, cada número se parte en su mitad alta y su mitad baja:

$$x = a \cdot 10^m + b, \qquad y = c \cdot 10^m + d.$$

Para 1234 y 5678 con $m = 2$: $a = 12$, $b = 34$, $c = 56$, $d = 78$. Multiplicando,

$$x\,y = ac \cdot 10^{2m} + (ad + bc) \cdot 10^m + bd.$$

Ejemplo 9.2.

$$\begin{aligned} ac &= 12 \cdot 56 = 672, & ad &= 12 \cdot 78 = 936, \\ bc &= 34 \cdot 56 = 1904, & bd &= 34 \cdot 78 = 2652. \end{aligned}$$

$$x\,y = 672 \cdot 10^4 + (936 + 1904) \cdot 10^2 + 2652 = 6\,720\,000 + 284\,000 + 2\,652 = 7\,006\,652.$$

Multiplicar por 10^k es agregar k ceros, un corrimiento que cuesta $\Theta(n)$, y las sumas también son $\Theta(n)$. Lo caro son las cuatro multiplicaciones de números de $n/2$ dígitos, que se resuelven recursivamente:

$$T(n) = 4\,T(n/2) + \Theta(n)\,.$$

Con $a = 4$ y $b = 2$, $n^{\log_2 4} = n^2$ crece polinomialmente más rápido que $f(n) = n$: caso 1 del método maestro, $T(n) = \Theta(n^2)$.

Ojito...

Dividir no ahorró nada. Cuatro problemas de la mitad del tamaño cuestan $4 \cdot (n/2)^2 = n^2$, lo mismo que el original. Divide y vencerás no es más rápido por partir el problema; es más rápido cuando los subproblemas, juntos, son menos trabajo que el original. Para eso hay que reducir *cuántos* son.

El truco de Karatsuba

En 1960 Kolmogorov propuso en un seminario que multiplicar no se podía hacer en menos de $\Theta(n^2)$. Anatoli Karatsuba, un estudiante de 23 años, encontró en una semana cómo hacerlo con tres multiplicaciones en lugar de cuatro.

La observación es que no hacen falta ad y bc por separado, solo su suma. Y

$$(a+b)(c+d) = ac + ad + bc + bd,$$

así que

$$ad + bc = (a+b)(c+d) - ac - bd.$$

ac y bd ya se iban a calcular de todos modos. Basta con tres multiplicaciones:

$$p_1 = ac, \qquad p_2 = bd, \qquad p_3 = (a+b)(c+d),$$

$$x\,y = p_1 \cdot 10^{2m} + (p_3 - p_1 - p_2) \cdot 10^m + p_2.$$

Ejemplo 9.3 (1234×5678 otra vez)**.** $p_1 = 672$ y $p_2 = 2652$ como antes. El tercero:

$$p_3 = (12+34)(56+78) = 46 \cdot 134 = 6164,$$

$$p_3 - p_1 - p_2 = 6164 - 672 - 2652 = 2840,$$

el mismo $ad + bc = 936 + 1904$ de antes, con una multiplicación menos. El resultado sale igual: 7 006 652.

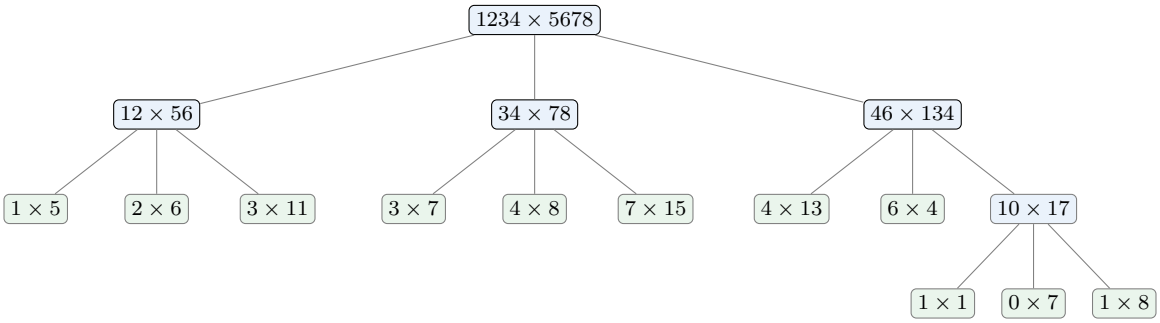
Se cambió una multiplicación por tres sumas y restas extra. Las sumas cuestan $\Theta(n)$ y las multiplicaciones mucho más, así que el cambio conviene.

Algoritmo 9.1 Karatsuba(x, y)

si $x < 10$ **o** $y < 10$ **entonces**
  devolver $x \cdot y$ *// un factor de un dígito: directo*
fin si
$n \leftarrow$ número de dígitos del mayor de x y y
$m \leftarrow \lfloor n/2 \rfloor$
$a \leftarrow \lfloor x/10^m \rfloor$, $\quad b \leftarrow x \bmod 10^m$
$c \leftarrow \lfloor y/10^m \rfloor$, $\quad d \leftarrow y \bmod 10^m$
$p_1 \leftarrow$ Karatsuba(a, c)
$p_2 \leftarrow$ Karatsuba(b, d)
$p_3 \leftarrow$ Karatsuba$(a+b,\ c+d)$
devolver $p_1 \cdot 10^{2m} + (p_3 - p_1 - p_2) \cdot 10^m + p_2$

La recursión completa de 1234×5678

Cada producto se parte en tres hasta que uno de los factores tiene un solo dígito. En verde, los casos base:



En cada nodo, los tres hijos son ac, bd y $(a+b)(c+d)$, en ese orden. Los nodos que no son base se arman así, de abajo hacia arriba:

Producto	m	$a, b \mid c, d$	p_1	p_2	p_3	$p_3 - p_1 - p_2$	**Resultado**
$10 \cdot 17$	1	$1, 0 \mid 1, 7$	1	0	8	7	$100 + 70 + 0 = 170$
$12 \cdot 56$	1	$1, 2 \mid 5, 6$	5	12	33	16	$500 + 160 + 12 = 672$
$34 \cdot 78$	1	$3, 4 \mid 7, 8$	21	32	105	52	$2100 + 520 + 32 = 2652$
$46 \cdot 134$	1	$4, 6 \mid 13, 4$	52	24	170	94	$5200 + 940 + 24 = 6164$
$1234 \cdot 5678$	2	$12, 34 \mid 56, 78$	672	2652	6164	2840	7 006 652

Ojito...

$a + b$ puede tener un dígito más que a y b: $56 + 78 = 134$. Por eso aparece $46 \cdot 134$, con factores de tamaños distintos, y por eso la rama de la derecha baja un nivel más. No cambia el análisis: el tercer subproblema es de tamaño $n/2 + 1$ en lugar de $n/2$, y ese $+1$ no altera el orden de crecimiento.

Observación. Con números de 4 dígitos Karatsuba no es más rápido que la escuela: hace casi las mismas multiplicaciones de un dígito y además sumas, restas y llamadas recursivas. La ventaja aparece con números grandes, que es lo que dice el análisis.

Análisis

Tres subproblemas de la mitad del tamaño, más sumas, restas y corrimientos lineales:

$$T(n) = 3\,T(n/2) + \Theta(n)\,.$$

Con $a = 3$, $b = 2$ y $f(n) = n$:

$$n^{\log_b a} = n^{\log_2 3} = n^{1.585}.$$

$f(n) = n = \mathcal{O}(n^{1.585-\varepsilon})$ con $\varepsilon \approx 0.585$, así que es el caso 1 del método maestro:

$$T(n) = \Theta\left(n^{\log_2 3}\right) \approx \Theta\left(n^{1.585}\right).$$

Con el árbol de recursión

Nivel	Subproblemas	Tamaño de cada uno	Trabajo del nivel
0	1	n	cn
1	3	$n/2$	$3 \cdot cn/2 = \frac{3}{2}cn$
2	9	$n/4$	$9 \cdot cn/4 = \left(\frac{3}{2}\right)^2 cn$
i	3^i	$n/2^i$	$\left(\frac{3}{2}\right)^i cn$
$\log_2 n$	$3^{\log_2 n} = n^{\log_2 3}$	1	$n^{\log_2 3}$

El trabajo crece $3/2$ veces en cada nivel, así que manda el último: las hojas. El árbol tiene $\log_2 n$ niveles y en el último hay $3^{\log_2 n} = n^{\log_2 3}$ casos base. Con cuatro subproblemas serían $4^{\log_2 n} = n^2$ hojas; pasar de 4 a 3 es lo que cambia el exponente.

Cuánto se gana

n (dígitos)	n^2	$n^{1.585}$	Karatsuba es
10	100	38	2.6 veces menos
100	10 000	1 479	6.8 veces menos
1 000	1 000 000	56 871	17.6 veces menos
1 000 000	10^{12}	3.2×10^9	309 veces menos

La ventaja crece con n sin límite, porque la razón entre los dos es $n^{2-1.585} = n^{0.415}$.

Ojito...

La tabla ignora las constantes. Karatsuba hace más sumas, restas y llamadas por nivel, y con números chicos eso pesa más que la multiplicación que se ahorra. Por eso las implementaciones reales, como las de Python o de la biblioteca GMP, usan la escuela para números chicos y solo cambian a Karatsuba a partir de cierto tamaño, del orden de cientos de dígitos. Además trabajan en binario: se parte por bits, y multiplicar por 2^m es un corrimiento de bits.

La lección general

Idea clave

En $T(n) = a\,T(n/b) + f(n)$, cuando dividir y combinar son baratos, manda $n^{\log_b a}$, y lo que más importa es a: *cuántos* subproblemas se generan. Gastar un poco más en combinar para generar un subproblema menos es el truco que mejora el exponente.

El mismo truco aparece en otros problemas:

- **Strassen** (1969), para multiplicar matrices de $n \times n$: partidas en bloques de la mitad, el método directo hace 8 productos de bloques, $\Theta(n^3)$. Strassen lo hace con 7 y queda $\Theta\left(n^{\log_2 7}\right) \approx \Theta(n^{2.807})$.
- **Toom-Cook**: parte cada número en tres partes y se las arregla con 5 multiplicaciones en lugar de 9: $\Theta\left(n^{\log_3 5}\right) \approx \Theta(n^{1.465})$.
- Para números enormes se usan algoritmos basados en la transformada rápida de Fourier, como el de Schönhage y Strassen, $\mathcal{O}(n \log n \log\log n)$. En 2019 Harvey y van der Hoeven llegaron a $\mathcal{O}(n \log n)$, que se cree que es lo mejor posible.

10. Programación dinámica

15 de octubre de 2026

Resumen rápido. Programación dinámica resuelve un problema combinando soluciones de subproblemas, como divide y vencerás, pero para el caso en que los subproblemas se repiten: cada uno se resuelve una sola vez y su resultado se guarda en una tabla. Hace falta que el problema tenga subestructura óptima y subproblemas superpuestos. Se ve con Fibonacci, que pasa de exponencial a lineal; con el cambio de monedas, donde el voraz falla; y a fondo con la mochila 0/1, la que quedó pendiente en la clase del 8: el voraz daba 160 y la programación dinámica encuentra el óptimo, 220.

La idea

Divide y vencerás parte un problema en subproblemas **independientes**: en Karatsuba, ac, bd y $(a+b)(c+d)$ son tres multiplicaciones distintas que nunca se vuelven a pedir. En muchos problemas no es así. Al partir, los mismos subproblemas aparecen una y otra vez, y la recursión los resuelve desde cero cada vez.

Definición 10.1 (Programación dinámica)**.** Una técnica para resolver problemas, casi siempre de optimización, combinando soluciones de subproblemas, en la que cada subproblema distinto se resuelve **una sola vez** y su solución se guarda en una tabla para consultarla cada vez que se vuelva a necesitar.

Se aplica cuando el problema tiene dos propiedades:

- **Subestructura óptima.** Una solución óptima del problema contiene soluciones óptimas de sus subproblemas. Eso permite armar la solución grande a partir de las chicas.
- **Subproblemas superpuestos.** La recursión natural visita los mismos subproblemas muchas veces. Si no se repitieran, guardarlos no ahorraría nada, y bastaría con divide y vencerás.

Observación. El nombre es de Richard Bellman, en los años cincuenta. "Programación" no se refiere a escribir código sino a planear con tablas, como en "programación lineal". "Dinámica" porque las decisiones se toman en etapas, una tras otra.

	Divide y vencerás	**Voraz**	**Programación dinámica**
Subproblemas	independientes, se resuelven todos	queda uno solo, el que deja la decisión	se repiten; cada uno se resuelve una vez
Decisión	no hay que elegir	se elige lo mejor del momento y no se revisa	se prueban todas las opciones y se elige la mejor
Ejemplo	mergesort, Karatsuba	mochila fraccional	mochila 0/1

Primer ejemplo: Fibonacci

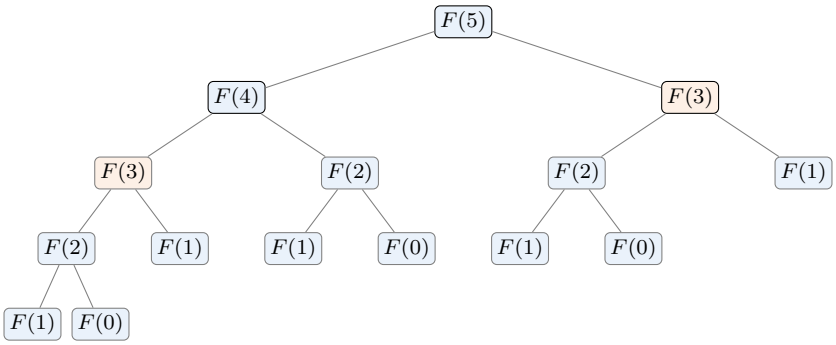
$$F(0) = 0, \qquad F(1) = 1, \qquad F(n) = F(n-1) + F(n-2).$$

La recursión directa copia la definición:

Algoritmo 10.1 Fib(n)

```
1: si n ≤ 1 entonces
2:     devolver n
3: fin si
4: devolver Fib(n − 1) + Fib(n − 2)
```

Para calcular $F(5)$ hace este árbol de llamadas:



Son 15 llamadas para obtener $F(5) = 5$, y casi todas repiten trabajo: $F(3)$ se calcula dos veces (en naranja), $F(2)$ tres veces, $F(1)$ cinco. Pero solo hay 6 subproblemas distintos, de $F(0)$ a $F(5)$.

El número de llamadas crece igual que la propia sucesión: para $F(n)$ son $2F(n+1) - 1$, que es $\Theta(\varphi^n)$ con $\varphi = (1 + \sqrt{5})/2 \approx 1.618$. Para $F(30) = 832\,040$ la recursión hace $2\,692\,537$ llamadas, cuando bastaba con 31 cálculos.

Dos formas de no repetir

Memoización, de arriba hacia abajo. Se deja la recursión igual, pero antes de calcular algo se revisa si ya está en una tabla M, y al calcularlo se guarda:

Algoritmo 10.2 Fib-Memo(n), con M inicializada en nulo

```
1: si M[n] ≠ nulo entonces
2:     devolver M[n]                          // ya se calculó: no se repite
3: fin si
4: si n ≤ 1 entonces
5:     M[n] ← n
6: si no
7:     M[n] ← Fib-Memo(n − 1) + Fib-Memo(n − 2)
8: fin si
9: devolver M[n]
```

Tabulación, de abajo hacia arriba. Se llena la tabla desde los casos base, en un orden en el que todo lo que hace falta ya está calculado:

Algoritmo 10.3 Fib-Tabla(n)

```
F[0] ← 0,    F[1] ← 1
para i ← 2 hasta n hacer
    F[i] ← F[i − 1] + F[i − 2]
fin para
devolver F[n]
```

i	0	1	2	3	4	5	6	7	8	9	10
$F[i]$	0	1	1	2	3	5	8	13	21	34	55

Las dos son $\Theta(n)$: cada uno de los $n+1$ subproblemas se resuelve una vez, en tiempo constante. La tabulación además deja ver que cada valor solo necesita los dos anteriores, así que basta con guardar dos números: espacio $\mathcal{O}(1)$.

Idea clave

El costo de un algoritmo de programación dinámica es casi siempre

$$\text{(subproblemas distintos)} \times \text{(trabajo por subproblema)},$$

donde el trabajo de cada uno se cuenta suponiendo que los demás ya están en la tabla. Para Fibonacci: $n+1$ subproblemas por $\mathcal{O}(1)$ cada uno.

El método en cuatro pasos

1. **Caracterizar la estructura de una solución óptima.** Qué decisión se toma y qué subproblema queda después de tomarla. De aquí sale qué hay que saber para describir un subproblema: el **estado**.

2. **Definir el valor óptimo de forma recursiva**, en términos de los subproblemas. Es la recurrencia.

3. **Calcular los valores de abajo hacia arriba**, llenando una tabla con un subproblema por casilla.

4. **Reconstruir la solución** con la información de la tabla: no solo cuánto vale el óptimo, sino cuál es.

Segundo ejemplo: cambio de monedas

Con monedas de 1, 3 y 4 pesos, ¿cuál es el menor número de monedas que suman 6? El voraz de la clase del 8 toma siempre la moneda más grande que quepa: $4+1+1$, tres monedas. Pero $3+3$ son dos.

Paso 1. La última moneda que se usa es de 1, de 3 o de 4. Si fue la de c pesos, lo que falta es pagar $a-c$ con el menor número de monedas: el mismo problema, más chico. El estado es solo la cantidad que falta pagar.

Paso 2. Sea $C[a]$ el menor número de monedas para pagar a:

$$C[0] = 0, \qquad C[a] = 1 + \min_{\substack{c \in \{1,3,4\} \\ c \le a}} C[a-c].$$

Se prueba cada moneda como la última y se queda la que deja el resto más barato.

Paso 3. De $a = 0$ a 6:

a	**Última** 1	**Última** 3	**Última** 4	$C[a]$	**Mejor última**
1	$1 + C[0] = 1$	—	—	1	1
2	$1 + C[1] = 2$	—	—	2	1
3	$1 + C[2] = 3$	$1 + C[0] = 1$	—	1	3
4	$1 + C[3] = 2$	$1 + C[1] = 2$	$1 + C[0] = 1$	1	4
5	$1 + C[4] = 2$	$1 + C[2] = 3$	$1 + C[1] = 2$	2	1
6	$1 + C[5] = 3$	$1 + C[3] = 2$	$1 + C[2] = 3$	2	3

Paso 4. Para 6 la mejor última moneda es la de 3; quedan 3, cuya mejor última es otra de 3; queda 0. Solución: $3 + 3$.

En $a = 6$ la tabla muestra justo dónde se equivoca el voraz: tomar 4 deja 2 pesos, que cuestan dos monedas; tomar 3 deja 3, que cuestan una. El voraz no compara, la tabla sí.

La mochila 0/1

Definición 10.2. Hay n objetos; el objeto i pesa w_i y vale v_i, y la mochila aguanta un peso W. Cada objeto se toma **completo o no se toma**: $x_i \in \{0, 1\}$. Se busca

$$\text{máx} \sum_{i=1}^{n} v_i\, x_i \qquad \text{sujeto a} \qquad \sum_{i=1}^{n} w_i\, x_i \le W.$$

Por qué no basta con lo que ya se tiene

- **Voraz.** En la clase del 8, con $W = 50$ y objetos de pesos $10, 20, 30$ y valores $60, 100, 120$, el voraz por densidad toma los dos primeros y vale 160; el tercero ya no cabe completo y sobran 20 de espacio. El óptimo es tomar el segundo y el tercero: 220.
- **Fuerza bruta.** Probar todos los subconjuntos de objetos funciona, pero son 2^n. Con 50 objetos son más de 10^{15}.

Para que la tabla quede chica, el ejemplo de esta sección es el mismo de la clase del 8 con los pesos divididos entre 10. Las decisiones no cambian:

Objeto i	**Peso** w_i	**Valor** v_i	**Densidad**
1	1	60	60
2	2	100	50
3	3	120	40

capacidad $W = 5$.

Paso 1: la estructura de una solución óptima

Hay que fijarse en una sola decisión: qué pasa con el **último** objeto, el n. En una solución óptima, o está o no está.

- **Si no está**, el resto de la solución es una forma de llenar la misma mochila de capacidad W con los objetos 1 a $n - 1$, y tiene que ser la mejor posible.
- **Si está**, ya ocupa w_n de espacio y aporta v_n. El resto es una forma de llenar el espacio que sobra, $W - w_n$, con los objetos 1 a $n - 1$, y también tiene que ser la mejor posible.

¿Por qué "tiene que ser la mejor posible"? Porque si el resto no fuera óptimo, se podría cambiar por uno mejor que pese lo mismo o menos, y la solución completa mejoraría. Eso contradice que era óptima. Es la subestructura óptima, y se demuestra siempre así: cortando la parte que no es óptima y pegando una mejor.

Idea clave

Aquí está la clave de todo el problema. Después de decidir sobre un objeto, el subproblema que queda no es solo "los objetos que faltan": también importa **cuánto espacio queda**. Por eso el estado necesita dos números,

$$(i,\ w) = (\text{cuántos objetos se pueden usar},\ \text{cuánta capacidad hay}).$$

El voraz falla justo por ignorar el segundo: decide sin preguntarse qué tan bien se va a poder aprovechar el espacio que deja.

Paso 2: la recurrencia

Sea

$$K[i][w] = \text{el mayor valor que se puede obtener usando solo los objetos } 1,\dots,i \text{ con capacidad } w.$$

La respuesta al problema es $K[n][W]$.

Casos base. Sin objetos, o sin capacidad, no se puede meter nada:

$$K[0][w] = 0, \qquad K[i][0] = 0.$$

Caso general. Para el objeto i hay dos opciones:

$$K[i][w] = \begin{cases} K[i-1][w] & \text{si } w_i > w \quad \text{(no cabe)}, \\ \max\Big(\underbrace{K[i-1][w]}_{\text{no tomarlo}},\ \underbrace{v_i + K[i-1][w-w_i]}_{\text{tomarlo}}\Big) & \text{si } w_i \le w. \end{cases}$$

- **No tomarlo**: la capacidad sigue siendo w y quedan los objetos anteriores. Vale lo mismo que la mejor solución de $1,\dots,i-1$ con capacidad w: la casilla de **arriba**.
- **Tomarlo**: se gana v_i, se pierden w_i de capacidad, y lo que sobra se llena lo mejor posible con los anteriores. Es la casilla del renglón de arriba, w_i **columnas a la izquierda**, más v_i.

Ojito...

Programar esta recurrencia tal cual, con recursión, hace dos llamadas por objeto: 2^n, lo mismo que la fuerza bruta. Pero solo hay $(n+1)(W+1)$ pares (i,w) distintos, así que con n grande y W chico casi todas las llamadas repiten un estado que ya se calculó. Son los subproblemas superpuestos, y la tabla los aprovecha.

Paso 3: llenar la tabla

La tabla tiene un renglón por cada $i = 0,\dots,3$ y una columna por cada capacidad $w = 0,\dots,5$. Cada casilla depende solo del renglón de arriba, así que se llena renglón por renglón.

Renglón 0. Sin objetos, todo vale 0.

Renglón 1, objeto 1 con peso 1 y valor 60. En $w = 0$ no cabe y queda 0. Desde $w = 1$ cabe, y tomarlo da $60 + K[0][w-1] = 60 + 0 = 60$ contra 0 de no tomarlo. Renglón: $0, 60, 60, 60, 60, 60$.

Renglón 2, objeto 2 con peso 2 y valor 100:

w	**¿Cabe?**	**No tomar** $K[1][w]$	**Tomar** $100 + K[1][w-2]$	$K[2][w]$	**Qué pasó**
0	no	0	—	0	se copia
1	no	60	—	60	se copia
2	sí	60	$100 + 0 = 100$	100	el 2 solo gana
3	sí	60	$100 + 60 = 160$	160	caben los dos
4	sí	60	$100 + 60 = 160$	160	igual
5	sí	60	$100 + 60 = 160$	160	igual

Renglón 3, objeto 3 con peso 3 y valor 120:

w	**¿Cabe?**	**No tomar** $K[2][w]$	**Tomar** $120 + K[2][w-3]$	$K[3][w]$	**Qué pasó**
0	no	0	—	0	se copia
1	no	60	—	60	se copia
2	no	100	—	100	se copia
3	sí	160	$120 + 0 = 120$	160	mejor 1 y 2
4	sí	160	$120 + 60 = 180$	180	mejor 3 y 1
5	sí	160	$120 + 100 = 220$	220	mejor 3 y 2

Vale la pena leer despacio las tres últimas filas. En $w = 3$ el objeto 3 cabe, pero tomarlo deja 0 de espacio y vale 120, menos que los objetos 1 y 2 juntos. En $w = 4$ tomarlo deja 1 de espacio, que el renglón de arriba sabe aprovechar con el objeto 1: $120 + 60 = 180$. En $w = 5$ deja 2, y lo mejor para 2 de espacio es el objeto 2: $120 + 100 = 220$.

La tabla completa, con la casilla de la respuesta y las dos de las que depende:

$i \setminus w$	0	1	2	3	4	5	
0	0	0	0	0	0	0	
1	0	60	60	60	60	60	objeto 1: peso 1, valor 60
2	0	60	100	160	160	160	objeto 2: peso 2, valor 100
3	0	60	100	160	180	220	objeto 3: peso 3, valor 120

$K[2][5] = 160$: no tomar el objeto 3 (la casilla de arriba)
$K[2][2] = 100$: tomarlo ($w_3 = 3$ columnas a la izquierda), $100 + 120 = 220$
$K[3][5] = \text{máx}(160,\ 220) = 220$: la respuesta

$K[3][5]$ compara la casilla de arriba, 160, con la que está $w_3 = 3$ columnas a la izquierda en el renglón de arriba, 100, más el valor del objeto, 120. Gana tomarlo: 220.

Paso 4: reconstruir qué objetos se tomaron

La tabla dice cuánto vale el óptimo, no cuáles objetos lo forman. Se recupera de atrás hacia adelante, empezando en $K[n][W]$. En cada renglón se pregunta si la casilla cambió respecto a la de arriba:

- Si $K[i][w] = K[i-1][w]$, el objeto i no hizo falta: se sube un renglón con la misma w.
- Si $K[i][w] \neq K[i-1][w]$, el valor solo se pudo alcanzar tomando el objeto i: se anota, se sube un renglón y se le restan w_i a la capacidad.

Paso	Casilla	Valor	Arriba	¿Cambió?	Decisión
1	$K[3][5]$	220	$K[2][5] = 160$	sí	se toma el 3; $w = 5 - 3 = 2$
2	$K[2][2]$	100	$K[1][2] = 60$	sí	se toma el 2; $w = 2 - 2 = 0$
3	$K[1][0]$	0	$K[0][0] = 0$	no	el 1 no

Se toman los objetos 2 y 3: pesan $2 + 3 = 5$, justo la capacidad, y valen $100 + 120 = 220$. Con los pesos originales es la respuesta de la clase del 8: los objetos de 20 y 30, con valor 220.

$i \setminus w$	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	60	60	60	60	60
2	0	60	100	160	160	160
3	0	60	100	160	180	220

camino: $K[3][5] \to K[2][2] \to K[1][0]$

En verde, el camino de la reconstrucción. De $K[3][5]$ a $K[2][2]$ se sube un renglón y se recorren $w_3 = 3$ columnas a la izquierda: se tomó el objeto 3. De $K[2][2]$ a $K[1][0]$, otro renglón y $w_2 = 2$ columnas: se tomó el 2. En $K[1][0]$ la casilla es igual a la de arriba, así que el objeto 1 no se tomó. Cada objeto tomado es un salto en diagonal; uno que no se toma sería un paso recto hacia arriba.

El algoritmo

Algoritmo 10.4 Mochila-01(w, v, n, W)

```
1: para c ← 0 hasta W hacer
2:     K[0][c] ← 0
3: fin para
4: para i ← 1 hasta n hacer
5:     para c ← 0 hasta W hacer
6:         K[i][c] ← K[i − 1][c]                          // no tomarlo
7:         si w_i ≤ c y v_i + K[i − 1][c − w_i] > K[i][c] entonces
8:             K[i][c] ← v_i + K[i − 1][c − w_i]          // tomarlo
9:         fin si
10:    fin para
11: fin para
12: devolver K
```

Algoritmo 10.5 Reconstruir(K, w, n, W)

```
1: S ← ∅,    c ← W
2: para i ← n hasta 1, de bajada hacer
3:     si K[i][c] ≠ K[i − 1][c] entonces
4:         S ← S ∪ {i},    c ← c − w_i
5:     fin si
6: fin para
7: devolver S
```

La tabla tiene $(n+1)(W+1)$ casillas y cada una se llena en tiempo constante: Mochila-01 es $\Theta(nW)$ en tiempo y en espacio. Reconstruir sube un renglón por paso: $\Theta(n)$.

Por qué $\Theta(nW)$ no es polinomial

Parece polinomial, pero W no es el tamaño de la entrada sino un número de la entrada. Escribir W ocupa $\log_2 W$ bits, y agregarle un solo bit duplica W y duplica la tabla. Respecto al tamaño de la entrada el algoritmo es exponencial; a esto se le llama **pseudopolinomial**.

Ojito...

Con $n = 100$ objetos y pesos en gramos hasta $W = 5\,000$, la tabla tiene medio millón de casillas: instantáneo. Con $W = 10^9$ tendría 10^{11} y ya no cabe en memoria. La mochila 0/1 es NP-difícil: no se conoce un algoritmo polinomial en el tamaño de la entrada, y la programación dinámica sirve cuando la capacidad es un número chico.

Segundo ejemplo, para practicar

Capacidad $W = 7$ y cuatro objetos:

Objeto	**A**	**B**	**C**	**D**
Peso	1	3	4	5
Valor	1	4	5	7
Densidad	1.00	1.33	1.25	1.40

El voraz por densidad toma D, el más denso; quedan 2 de espacio, donde ni B ni C caben, y toma A. Vale $7 + 1 = 8$.

La tabla de programación dinámica, con los objetos en el orden A, B, C, D:

$i \setminus w$	0	1	2	3	4	5	6	7
0 (ninguno)	0	0	0	0	0	0	0	0
1 (A: 1, 1)	0	1	1	1	1	1	1	1
2 (B: 3, 4)	0	1	1	4	5	5	5	5
3 (C: 4, 5)	0	1	1	4	5	6	6	**9**
4 (D: 5, 7)	0	1	1	4	5	7	8	**9**

La casilla clave es la última:

$$K[4][7] = \text{máx}\,(\underbrace{K[3][7]}_{\text{no tomar D}},\ \underbrace{7 + K[3][2]}_{\text{tomar D}}) = \text{máx}(9,\ 7+1) = 9.$$

Tomar D, lo que haría el voraz, deja 2 de espacio que solo valen 1. No tomarlo deja los 7 de capacidad para los objetos anteriores, que valen 9.

Reconstrucción: $K[4][7] = K[3][7]$, así que D no se toma. $K[3][7] = 9 \neq K[2][7] = 5$: se toma C y quedan $7 - 4 = 3$. $K[2][3] = 4 \neq K[1][3] = 1$: se toma B y quedan 0. Solución: B y C, que pesan 7 y valen 9.

Ahorrar memoria: una sola fila

Cada renglón solo usa el de arriba, así que basta un arreglo de $W + 1$ casillas que se reescribe para cada objeto:

Algoritmo 10.6 Mochila-01-Una-Fila(w, v, n, W)

```
1: K[0 . . . W] ← 0
2: para i ← 1 hasta n hacer
3:     para c ← W hasta w_i, de bajada hacer
4:         K[c] ← máx (K[c], v_i + K[c − w_i])
5:     fin para
6: fin para
7: devolver K[W]
```

El espacio baja a $\Theta(W)$. Pero el orden del ciclo interno importa: va **de bajada**.

Ojito...

Al calcular $K[c]$ se lee $K[c - w_i]$, que debe ser el valor del renglón anterior, sin el objeto i. Recorriendo de bajada, $K[c - w_i]$ todavía no se ha reescrito. De subida ya se reescribió, y el objeto se podría tomar dos veces. Con el objeto 1 (peso 1, valor 60) y $W = 2$, de subida:

$$K[1] = \text{máx}(0,\ 60 + K[0]) = 60, \qquad K[2] = \text{máx}(0,\ 60 + K[1]) = 120,$$

como si hubiera dos copias del objeto. De bajada, $K[2]$ se calcula primero con $K[1] = 0$ y queda en 60. El recorrido de subida resuelve otro problema, la mochila **sin límite**, donde cada objeto se puede tomar cuantas veces se quiera.

Cómo reconocer un problema de programación dinámica

Idea clave

Las señales son tres: el problema pide un óptimo o un conteo; la solución se arma con una secuencia de decisiones; y al escribir la recursión, los mismos estados aparecen por distintos caminos. El trabajo difícil es el paso 1: encontrar el **estado**, lo mínimo que hay que saber para resolver lo que falta. En Fibonacci es un número; en las monedas, cuánto falta pagar; en la mochila, qué objetos quedan y cuánto espacio hay.

Problema	Estado	Recurrencia	Costo
Fibonacci	i	$F[i] = F[i-1] + F[i-2]$	$\Theta(n)$
Monedas	cantidad a	$C[a] = 1 + \min_c C[a-c]$	$\Theta(a \cdot k)$, k monedas
Mochila 0/1	objetos i, capacidad w	$\text{máx}(K[i-1][w],\ v_i + K[i-1][w-w_i])$	$\Theta(nW)$

11. Grafos

22 de octubre de 2026

Resumen rápido. Un grafo es un par de conjuntos, vértices y aristas, y modela cualquier cosa hecha de objetos y relaciones entre pares de ellos: ciudades y carreteras, páginas y ligas, estados de un juego y jugadas. Se representa con matriz o con listas de adyacencia, y se clasifica por sus aristas: dirigido o no, pesado o no, cíclico o acíclico, simple o no, encajado o topológico, implícito o explícito. Se vieron los dos recorridos, búsqueda en anchura (con cola) y en profundidad (con recursión o pila), y tres problemas: camino más corto con Dijkstra y Floyd, detección de ciclos con BFS y árbol de expansión mínima con Kruskal. Todos los ejemplos usan los mismos tres grafos.

Definición formal

Definición 11.1 (Grafo)**.** Un **grafo** es un par $G = (V, E)$ donde V es un conjunto finito de **vértices** y E un conjunto de **aristas**. En un grafo no dirigido cada arista es un par no ordenado de vértices distintos,

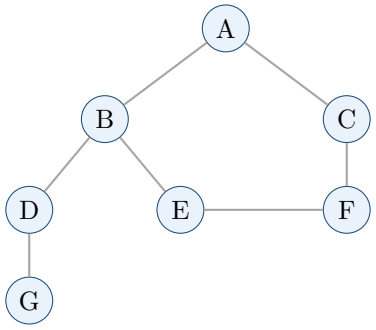
$$E \subseteq \{\{u, v\} : u, v \in V,\ u \neq v\}.$$

Se escribe $n = |V|$ y $m = |E|$.

Si $\{u, v\} \in E$, los vértices u y v son **adyacentes** y la arista es **incidente** en los dos. El **grado** de un vértice, $\mathrm{gr}(v)$, es el número de aristas incidentes en él.

Ejemplo 11.2 (El grafo G de esta clase)**.**

$$V = \{A, B, C, D, E, F, G\}, \qquad E = \{\{A, B\}, \{A, C\}, \{B, D\}, \{B, E\}, \{C, F\}, \{E, F\}, \{D, G\}\}.$$



$n = 7$ y $m = 7$. Los grados son $\mathrm{gr}(B) = 3$, $\mathrm{gr}(G) = 1$ y 2 para los demás. Este grafo se usa en toda la clase.

Observación. Cada arista suma 1 al grado de sus dos extremos, así que $\sum_{v \in V} \mathrm{gr}(v) = 2m$. En G: $2{+}3{+}2{+}2{+}2{+}2{+}1 = 14 = 2{\cdot}7$. Y como hay a lo más un par por cada dos vértices, $0 \leq m \leq n(n-1)/2$.

Matriz y listas de adyacencia

Definición 11.3 (Matriz de adyacencia)**.** Con los vértices numerados de 1 a n, la matriz de adyacencia es la matriz A de $n \times n$ con

$$A[i][j] = \begin{cases} 1 & \text{si } \{i, j\} \in E, \\ 0 & \text{si no.} \end{cases}$$

Para G, en el orden $A, \ldots, G$:

	A	B	C	D	E	F	G
A	0	1	1	0	0	0	0
B	1	0	0	1	1	0	0
C	1	0	0	0	0	1	0
D	0	1	0	0	0	0	1
E	0	1	0	0	0	1	0
F	0	0	1	0	1	0	0
G	0	0	0	1	0	0	0

En un grafo no dirigido la matriz es simétrica, la diagonal es cero y la suma de cada renglón es el grado del vértice.

La alternativa son las **listas de adyacencia**: para cada vértice, la lista de sus vecinos.

$A : B, C$
$B : A, D, E$
$C : A, F$
$D : B, G$
$E : B, F$
$F : C, E$
$G : D$

Operación	**Matriz**	**Listas**
Memoria	$\Theta(n^2)$	$\Theta(n+m)$
¿Es $\{u, v\}$ arista?	$\mathcal{O}(1)$	$\mathcal{O}(\text{gr}(u))$
Recorrer los vecinos de u	$\Theta(n)$	$\Theta(\text{gr}(u))$
Recorrer todo el grafo	$\Theta(n^2)$	$\Theta(n+m)$

Idea clave

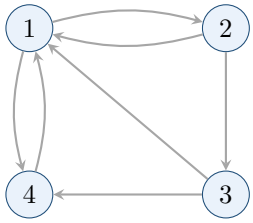
Casi todos los grafos reales son **dispersos**: m es mucho menor que n^2. Un mapa de carreteras con un millón de cruces tiene unos cuantos millones de tramos, no 10^{12}. Ahí la matriz gasta memoria en ceros y las listas ganan. La matriz conviene en grafos **densos** o cuando se pregunta mucho si una arista existe. Las complejidades de esta clase suponen listas.

Tipos de grafos

La clasificación es la de Skiena. Cada par describe una propiedad de las aristas o de cómo se guarda el grafo, y cambia qué algoritmo se puede usar.

Dirigido o no dirigido

En un **grafo dirigido** las aristas son pares ordenados, $E \subseteq V \times V$: la arista (u, v) va de u a v y no al revés. Cada vértice tiene **grado de entrada** y **grado de salida**, y la matriz de adyacencia deja de ser simétrica. Una calle de un solo sentido o *A sigue a B* en una red social son dirigidas; una amistad no.

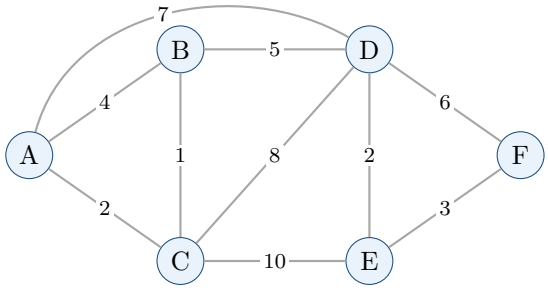


	1	2	3	4
1	0	1	0	1
2	1	0	1	0
3	1	0	0	1
4	1	0	0	0

$A[2][3] = 1$ pero $A[3][2] = 0$: de 2 se llega a 3 en un paso, de 3 a 2 no. El vértice 1 tiene grado de entrada 3 y de salida 2. Este grafo se usa en Floyd.

Pesado o no pesado

En un **grafo pesado** cada arista tiene un número, $w : E \to \mathbb{R}$: distancia, tiempo, costo. En la matriz se guarda el peso en lugar del 1, y ∞ donde no hay arista.

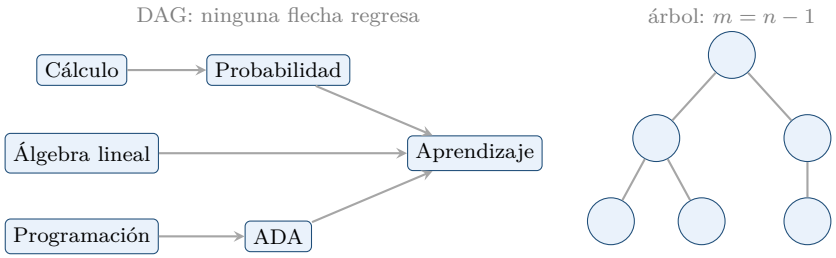


	A	B	C	D	E	F
A	0	4	2	7	∞	∞
B	4	0	1	5	∞	∞
C	2	1	0	8	10	∞
D	7	5	8	0	2	6
E	∞	∞	10	2	0	3
F	∞	∞	∞	6	3	0

En un grafo no pesado el camino más corto es el de menos aristas, y lo encuentra BFS. En uno pesado el de menos aristas puede no ser el más barato: de A a D la arista directa pesa 7, y A, C, B, D pesa $2 + 1 + 5 = 8$. Hacen falta Dijkstra o Floyd. Este grafo se usa en Dijkstra y en Kruskal.

Cíclico o acíclico

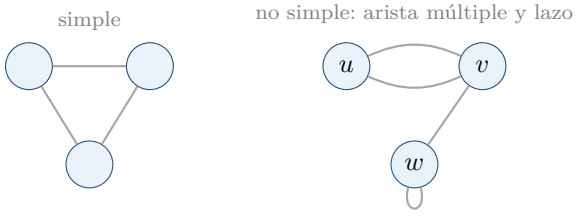
Un **ciclo** es un camino que regresa a su vértice de inicio sin repetir aristas. G tiene uno: A, B, E, F, C, A. Un grafo sin ciclos es **acíclico**. Un no dirigido, conexo y acíclico es un **árbol**; un dirigido acíclico es un **DAG** (*directed acyclic graph*), como las seriaciones de un plan de estudios.



Un DAG siempre tiene un **orden topológico**: una forma de poner los vértices en fila con todas las flechas hacia adelante, el orden en que se pueden cursar las materias. Un grafo con un ciclo dirigido no lo tiene.

Simple o no simple

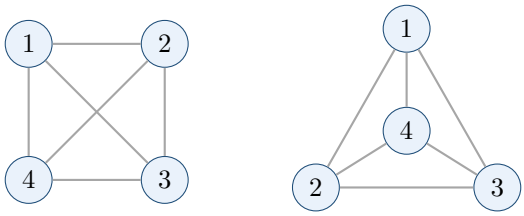
Un grafo es **simple** si no tiene **lazos** (aristas de un vértice a sí mismo) ni **aristas múltiples** (dos aristas entre el mismo par). La definición formal de arriba, con conjuntos, solo admite grafos simples: $\{u, u\}$ no es un par y un conjunto no repite elementos. Para los no simples E se vuelve un multiconjunto.



Una arista múltiple aparece, por ejemplo, con dos vuelos distintos entre las mismas ciudades. Casi todos los algoritmos suponen grafos simples: con aristas múltiples pesadas basta quedarse con la más barata.

Encajado o topológico

Un grafo es **encajado** (*embedded*) si a sus vértices y aristas se les asignan posiciones geométricas: un punto en el plano para cada vértice y una curva para cada arista. Un grafo es **topológico** si solo importa quién está unido con quién. Los dos dibujos de abajo son el mismo grafo topológico, K_4, con cuatro vértices todos unidos entre sí, en dos encajes distintos:

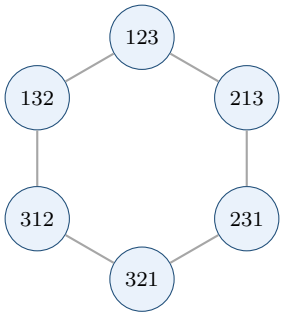


A la izquierda dos aristas se cruzan; a la derecha ninguna. Como grafo topológico es lo mismo; como grafo encajado no. En un mapa la geometría es parte del problema: la distancia entre dos cruces sale de sus coordenadas, y se puede saber si dos calles se cruzan.

Implícito o explícito

Un grafo **explícito** se construye completo y se guarda en una matriz o en listas antes de recorrerlo. Uno **implícito** no se guarda nunca: los vecinos de un vértice se generan cuando se necesitan, con una regla.

Ejemplo 11.4 (Permutaciones como grafo implícito)**.** Los vértices son las permutaciones de $1, 2, 3$, y dos son vecinas si una sale de la otra intercambiando dos elementos contiguos:



Los vecinos de 123 son 213 y 132, y se obtienen con la regla sin haber guardado nada. Con n elementos el grafo tiene $n!$ vértices: con $n = 10$ son $3\,628\,800$, y solo se generan los que el recorrido alcanza a visitar.

Los grafos implícitos aparecen en búsqueda: los estados de un rompecabezas, las posiciones de un juego, las soluciones parciales de un problema de backtracking.

Búsqueda en anchura (BFS)

Recorrer un grafo es visitar todos los vértices alcanzables desde uno inicial s, cada uno una sola vez. La **búsqueda en anchura** (*breadth-first search*) visita por capas: primero s, luego todos sus vecinos, luego los vecinos de esos que no se han visto, y así. Para respetar ese orden usa una **cola**: el primer vértice descubierto es el primero que se procesa.

Algoritmo 11.1 BFS(G, s)

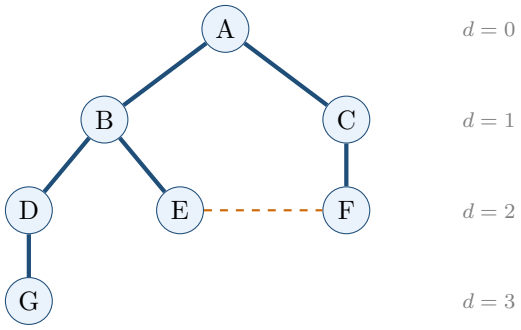
```
1:  para todo v ∈ V hacer
2:      visto[v] ← falso,    d[v] ← ∞,    π[v] ← nulo
3:  fin para
4:  visto[s] ← verdadero,    d[s] ← 0
5:  Q ← cola vacía;  ENCOLAR(Q, s)
6:  mientras Q no está vacía hacer
7:      u ← DESENCOLAR(Q)
8:      para todo v en la lista de adyacencia de u hacer
9:          si no visto[v] entonces
10:             visto[v] ← verdadero,    d[v] ← d[u] + 1,    π[v] ← u
11:             ENCOLAR(Q, v)
12:         fin si
13:     fin para
14: fin mientras
```

$d[v]$ es la distancia en aristas desde s y $\pi[v]$ el vértice desde el que se descubrió v, su **padre**.

Ejemplo 11.5 (BFS sobre G desde A)**.** Los vecinos se recorren en orden alfabético.

Paso	Sale	Descubre	Cola después
1	A	B, C	[B, C]
2	B	D, E (A ya visto)	[C, D, E]
3	C	F (A ya visto)	[D, E, F]
4	D	G (B ya visto)	[E, F, G]
5	E	nada: B y F ya vistos	[F, G]
6	F	nada: C y E ya vistos	[G]
7	G	nada	[ ]

Orden de visita: A, B, C, D, E, F, G. Las aristas por las que se descubrió cada vértice forman el **árbol BFS**:



Cada renglón es una capa: todos los vértices a la misma distancia de A. La arista E–F, punteada, no está en el árbol: cuando se procesó E, F ya estaba visto.

Cada vértice entra y sale de la cola una vez, $\Theta(n)$, y la lista de cada vértice se recorre una vez; las listas suman $2m$ entradas. En total BFS es $\Theta(n + m)$ con listas, y $\Theta(n^2)$ con matriz, porque encontrar los vecinos de cada vértice cuesta $\Theta(n)$.

Idea clave

En un grafo no pesado, $d[v]$ es la longitud del camino más corto de s a v, y siguiendo π de v hacia atrás se reconstruye ese camino. BFS ya resuelve el camino más corto cuando todas las aristas valen lo mismo.

Ojito...

En clase se dijo que la complejidad depende del problema, y que en un grafo completamente conectado es factorial. Hay que separar dos cosas. BFS visita cada vértice y cada arista una vez: en el grafo completo K_n, con $m = n(n-1)/2$, cuesta $\Theta(n^2)$, no factorial. Lo factorial aparece cuando el problema exige recorrer **caminos** y no vértices, como probar todas las rutas posibles: desde un vértice de K_5 salen 65 caminos simples y desde uno de K_{10}, 986 410, alrededor de $e \cdot (n-1)!$. O cuando el grafo es implícito y sus vértices son permutaciones, como el ejemplo de arriba: BFS sigue siendo lineal en el tamaño del grafo, pero el grafo mide $n!$.

Búsqueda en profundidad (DFS)

La **búsqueda en profundidad** (*depth-first search*) se mete lo más hondo que puede por un camino y solo regresa cuando se atora: cuando el vértice actual ya no tiene vecinos sin visitar. Se escribe de forma natural con recursión; la pila de llamadas hace el papel que la cola hace en BFS. La versión iterativa usa una pila explícita.

Algoritmo 11.2 DFS-Visitar(G, u)

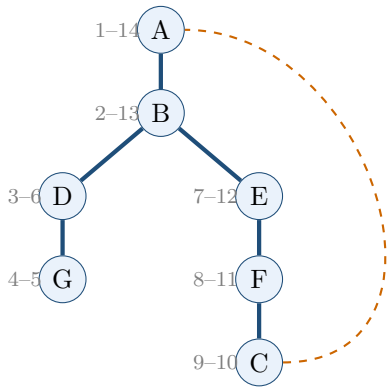
```
1: t ← t + 1,    entra[u] ← t,    visto[u] ← verdadero
2: para todo v en la lista de adyacencia de u hacer
3:     si no visto[v] entonces
4:         π[v] ← u
5:         DFS-Visitar(G, v)
6:     fin si
7: fin para
8: t ← t + 1,    sale[u] ← t
```

t es un reloj global que empieza en 0. entra$[u]$ marca cuándo se descubrió u y sale$[u]$ cuándo se terminó con todo lo que cuelga de él.

Ejemplo 11.6 (DFS sobre G desde A)**.** Con los vecinos en orden alfabético, como en BFS:

- A → B → D → G. G no tiene vecinos nuevos: se termina, y se regresa a D, que tampoco tiene; se regresa a B.
- B → E → F → C. Los vecinos de C son A y F, los dos vistos: se termina C, luego F, E, B y A.

Orden de visita: A, B, D, G, E, F, C. El de BFS era A, B, C, D, E, F, G.



Junto a cada vértice, entra–sale. El árbol DFS es largo y angosto; el BFS, ancho y bajo. La arista C–A, punteada, es una **arista de retroceso**: va de C a un antecesor suyo en el árbol.

DFS también cuesta $\Theta(n+m)$ con listas, por la misma cuenta que BFS.

	BFS	**DFS**
Estructura	cola	pila o recursión
Orden	por capas, de cerca a lejos	un camino hasta el fondo
Árbol	ancho y bajo	largo y angosto
Útil para	camino más corto sin pesos	ciclos dirigidos, orden topológico, componentes
Costo con listas	$\Theta(n+m)$	$\Theta(n+m)$

Camino más corto: Dijkstra

Dado un grafo pesado con pesos **no negativos** y un vértice s, el algoritmo de Dijkstra (1959) encuentra la distancia de s a todos los demás. Mantiene una estimación $d[v]$ que empieza en ∞, y en cada paso fija el vértice no fijado con la d más chica: esa ya es su distancia definitiva. Luego **relaja** las aristas que salen de él: si llegar a un vecino pasando por u es más barato que lo que se tenía, se actualiza.

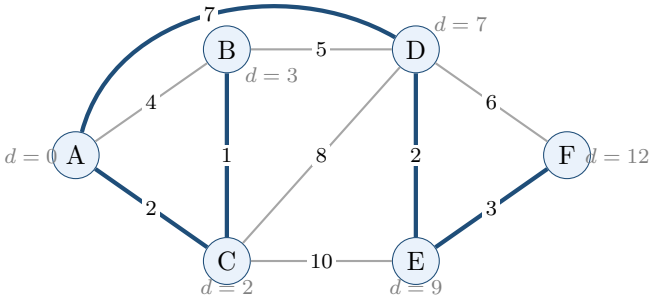
Algoritmo 11.3 Dijkstra(G, w, s)

para todo $v \in V$ **hacer**
  $d[v] \leftarrow \infty$, $\pi[v] \leftarrow$ nulo
fin para
$d[s] \leftarrow 0$
$Q \leftarrow$ montículo de mínimos con todos los vértices, con llave d
mientras Q no está vacío **hacer**
  $u \leftarrow$ Extraer-Min(Q) *// $d[u]$ queda fija*
  para todo v adyacente a u **hacer**
    si $d[u] + w(u,v) < d[v]$ **entonces** *// relajar*
      $d[v] \leftarrow d[u] + w(u,v)$, $\pi[v] \leftarrow u$
      Decrementar-Llave$(Q, v, d[v])$
    fin si
  fin para
fin mientras

Ejemplo 11.7 (Dijkstra sobre el grafo pesado desde A)**.** Cada renglón muestra las d después de procesar el vértice que sale; en negritas, las ya fijas.

Paso	Sale	A	B	C	D	E	F	Relajaciones
0		0	∞	∞	∞	∞	∞	
1	A	**0**	4	2	7	∞	∞	B, C, D desde A
2	C	**0**	3	**2**	7	12	∞	B: $2+1=3<4$; D: 10, no
3	B	**0**	**3**	**2**	7	12	∞	D: $3+5=8$, no mejora
4	D	**0**	**3**	**2**	**7**	9	13	E: $7+2=9<12$
5	E	**0**	**3**	**2**	**7**	**9**	12	F: $9+3=12<13$
6	F	**0**	**3**	**2**	**7**	**9**	**12**	

B empezó en 4 por la arista directa y bajó a 3 pasando por C. Los padres quedan $\pi[B]=C$, $\pi[C]=A$, $\pi[D]=A$, $\pi[E]=D$, $\pi[F]=E$, y el camino a F se lee de atrás hacia adelante: F $\leftarrow$ E $\leftarrow$ D $\leftarrow$ A, o sea A, D, E, F, con $7+2+3=12$.



En azul, el **árbol de caminos más cortos**: las aristas $\pi[v]$–v.

Por qué funciona. Cuando u sale con la d mínima, cualquier otro camino a u tendría que pasar por algún vértice no fijado, que ya tiene $d \geq d[u]$, y luego sumar aristas que no pueden restar. Es un algoritmo voraz: la elección del momento nunca se revisa, y con pesos no negativos no hace falta.

Costo. Con montículo binario, n extracciones de $\mathcal{O}(\log n)$ y hasta m decrementos de $\mathcal{O}(\log n)$: $\mathcal{O}((n+m)\log n)$. Es el montículo de la clase del 17 de septiembre. Con un arreglo, buscar el mínimo cuesta $\Theta(n)$ y el total es $\Theta(n^2)$, mejor cuando el grafo es denso.

Ojito...

Con un peso negativo Dijkstra falla. Con las aristas dirigidas $A \to B$ de peso 2, $A \to C$ de peso 3 y $C \to B$ de peso -2, desde A se fija B con $d=2$ antes de procesar C. Pero $A \to C \to B$ cuesta $3-2=1$. El argumento de arriba suponía que sumar aristas nunca resta.

Camino más corto entre todos los pares: Floyd

Dijkstra da las distancias desde un vértice. Para tener las de todos contra todos se podría correr n veces, pero Floyd (Floyd–Warshall, 1962) lo resuelve con programación dinámica en tres ciclos, y admite pesos negativos siempre que no haya ciclos de peso negativo.

Con los vértices numerados de 1 a n, sea $d_{ij}^{(k)}$ el camino más corto de i a j que solo usa como intermedios vértices de $\{1,\ldots,k\}$. Para pasar de $k-1$ a k hay dos opciones: el camino no usa k, y queda igual; o lo usa, y se parte en dos tramos que ya están calculados:

$$d_{ij}^{(k)} = \min\left(d_{ij}^{(k-1)},\ d_{ik}^{(k-1)} + d_{kj}^{(k-1)}\right),$$

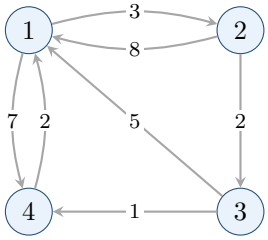
con $d_{ij}^{(0)} = w(i,j)$, 0 en la diagonal e ∞ si no hay arista. La respuesta es $D^{(n)}$.

Algoritmo 11.4 Floyd(W, n)

```
1: D ← W
2: para k ← 1 hasta n hacer
3:     para i ← 1 hasta n hacer
4:         para j ← 1 hasta n hacer
5:             D[i][j] ← mín(D[i][j], D[i][k] + D[k][j])
6:         fin para
7:     fin para
8: fin para
9: devolver D
```

Son $\Theta(n^3)$ operaciones y $\Theta(n^2)$ de memoria, porque la misma matriz se puede actualizar en su lugar.

Ejemplo 11.8 (Floyd sobre el grafo dirigido)**.** Los pesos de las aristas del grafo dirigido de arriba:



En negritas, las casillas que cambian en cada paso:

$$D^{(0)} = \begin{pmatrix} 0 & 3 & \infty & 7 \\ 8 & 0 & 2 & \infty \\ 5 & \infty & 0 & 1 \\ 2 & \infty & \infty & 0 \end{pmatrix} \qquad D^{(1)} = \begin{pmatrix} 0 & 3 & \infty & 7 \\ 8 & 0 & 2 & \mathbf{15} \\ 5 & \mathbf{8} & 0 & 1 \\ 2 & \mathbf{5} & \infty & 0 \end{pmatrix} \qquad D^{(2)} = \begin{pmatrix} 0 & 3 & \mathbf{5} & 7 \\ 8 & 0 & 2 & 15 \\ 5 & 8 & 0 & 1 \\ 2 & 5 & \mathbf{7} & 0 \end{pmatrix}$$

$$D^{(3)} = \begin{pmatrix} 0 & 3 & 5 & \mathbf{6} \\ \mathbf{7} & 0 & 2 & \mathbf{3} \\ 5 & 8 & 0 & 1 \\ 2 & 5 & 7 & 0 \end{pmatrix} \qquad D^{(4)} = \begin{pmatrix} 0 & 3 & 5 & 6 \\ \mathbf{5} & 0 & 2 & 3 \\ \mathbf{3} & \mathbf{6} & 0 & 1 \\ 2 & 5 & 7 & 0 \end{pmatrix}$$

Algunos cambios, uno por paso:

- $k = 1$: de 3 a 2 no había arista; pasando por 1, $d_{31} + d_{12} = 5 + 3 = 8$.
- $k = 2$: de 1 a 3, $d_{12} + d_{23} = 3 + 2 = 5$.
- $k = 3$: de 2 a 4 bajó de 15 a $d_{23} + d_{34} = 2 + 1 = 3$.
- $k = 4$: de 3 a 1 bajó de 5 a $d_{34} + d_{41} = 1 + 2 = 3$.

$D^{(4)}$ tiene todas las distancias. Por ejemplo $d_{32} = 6$ es el camino $3 \to 4 \to 1 \to 2$, con $1 + 2 + 3 = 6$.

	Dijkstra	**Floyd**
Responde	desde un vértice	todos los pares
Pesos negativos	no	sí, sin ciclos negativos
Técnica	voraz	programación dinámica
Costo	$\mathcal{O}((n+m)\log n)$	$\Theta(n^3)$

Encontrar ciclos con BFS

En un grafo no dirigido, BFS encuentra un ciclo en cuanto ve una arista que no es del árbol: al procesar u, un vecino v que ya fue visto y que no es el padre de u. El padre no cuenta porque la arista $\{u, \pi[u]\}$ es justo por la que se llegó a u. Basta cambiar el **si** de BFS:

Algoritmo 11.5 Cambio a BFS para detectar un ciclo

```
1: para todo v en la lista de adyacencia de u hacer
2:     si no visto[v] entonces
3:         ... igual que en BFS ...
4:     si no si v ≠ π[u] entonces
5:         devolver hay un ciclo que pasa por la arista {u, v}
6:     fin si
7: fin para
```

Ejemplo 11.9 (El ciclo de G)**.** En el paso 5 de BFS sale E y su vecino F ya está visto, pero $\pi[E] = B \neq F$. Hay ciclo. Para recuperarlo se sube por los padres desde los dos extremos hasta que se encuentran:

$$E \to B \to A, \qquad F \to C \to A.$$

Se juntan en A, y el ciclo es A, B, E, F, C, A: la arista punteada del árbol BFS cierra la vuelta.

Ojito...

En un grafo dirigido esta regla da falsos positivos. Con $A \to B$, $A \to C$ y $B \to C$, BFS desde A llega a C dos veces y la regla dice ciclo, pero no hay ningún ciclo dirigido. Para grafos dirigidos se usa DFS, donde un ciclo es una arista de retroceso, o el algoritmo de Kahn: con una cola se van quitando los vértices de grado de entrada 0; si alguno nunca llega a 0, hay ciclo. En el grafo de Floyd ningún vértice empieza con grado de entrada 0, así que la cola nace vacía: tiene ciclos, como $1 \to 2 \to 1$.

Árbol de expansión mínima: Kruskal

Definición 11.10 (Árbol de expansión mínima)**.** En un grafo no dirigido, conexo y pesado, un **árbol de expansión** es un subconjunto de aristas que conecta todos los vértices sin formar ciclos; tiene exactamente $n - 1$ aristas. El de **expansión mínima** (MST, *minimum spanning tree*) es el de menor peso total.

Es el problema de conectar n ciudades con cable gastando lo menos posible. El algoritmo de Kruskal (1956) es voraz: ordena las aristas de la más ligera a la más pesada y las agrega en ese orden, saltándose las que cerrarían un ciclo.

Algoritmo 11.6 Kruskal(G, w)

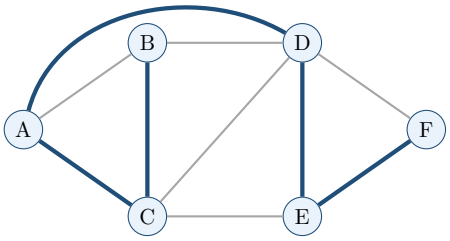
```
1:  T ← ∅
2:  para todo v ∈ V hacer
3:      Crear-Conjunto(v)
4:  fin para
5:  ordenar E por peso, de menor a mayor
6:  para todo {u, v} ∈ E, en ese orden hacer
7:      si Encontrar(u) ≠ Encontrar(v) entonces
8:          T ← T ∪ {{u, v}}
9:          Unir(u, v)
10:     fin si
11: fin para
12: devolver T
```

Para saber si una arista cierra un ciclo se usa una estructura de **conjuntos disjuntos** (*union-find*): cada vértice empieza en su propio conjunto, Encontrar dice en qué conjunto está un vértice y Unir junta dos. Si u y v ya están en el mismo conjunto, ya hay un camino entre ellos y la arista cerraría un ciclo.

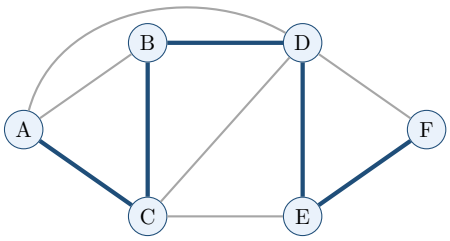
Ejemplo 11.11 (Kruskal sobre el grafo pesado)**.** Las aristas en orden de peso:

Arista	Peso	¿Entra?	Componentes después
B–C	1	sí	{A} {B, C} {D} {E} {F}
A–C	2	sí	{A, B, C} {D} {E} {F}
D–E	2	sí	{A, B, C} {D, E} {F}
E–F	3	sí	{A, B, C} {D, E, F}
A–B	4	no: A y B ya están juntos	
B–D	5	sí	{A, B, C, D, E, F}

Con $n - 1 = 5$ aristas el árbol está completo, y las que faltan (D–F, A–D, C–D, C–E) cerrarían ciclos. Peso total: $1 + 2 + 2 + 3 + 5 = 13$.



Caminos más cortos desde A: peso 15



Expansión mínima: peso 13

Ojito...

El árbol de caminos más cortos de Dijkstra y el árbol de expansión mínima no son lo mismo, aunque los dos tengan $n - 1$ aristas. El de Dijkstra minimiza la distancia de A a cada vértice; el MST, el peso total. En el MST el camino de A a D es A, C, B, D, con 8; en el de Dijkstra es la arista directa, con 7. Pero el MST pesa 13 y el de Dijkstra 15.

Por qué funciona. Si se parte el conjunto de vértices en dos lados, la arista más ligera que cruza de un lado al otro está en algún MST: cualquier árbol que use otra arista de cruce se puede mejorar

cambiándola por esa. Cada arista que Kruskal acepta es la más ligera entre dos componentes, así que siempre es segura. Es la misma idea voraz de la clase del 8 de octubre.

Costo. Ordenar las aristas cuesta $\mathcal{O}(m \log m)$. Con unión por rango y compresión de caminos, las $\mathcal{O}(m)$ operaciones de Encontrar y Unir cuestan prácticamente $\mathcal{O}(m)$. Domina el ordenamiento: $\mathcal{O}(m \log m) = \mathcal{O}(m \log n)$, porque $m \leq n^2$.

Observación. El otro algoritmo clásico es el de Prim: hace crecer un solo árbol desde un vértice, agregando siempre la arista más ligera que sale de él. Es casi idéntico a Dijkstra, cambiando la llave $d[u] + w(u, v)$ por solo $w(u, v)$.

Resumen

Algoritmo	**Resuelve**	**Estructura**	**Costo**
BFS	recorrido por capas; camino más corto sin pesos; ciclos en no dirigidos	cola	$\Theta(n + m)$
DFS	recorrido en profundidad; ciclos dirigidos; orden topológico	pila o recursión	$\Theta(n + m)$
Dijkstra	caminos más cortos desde un vértice, pesos ≥ 0	montículo	$\mathcal{O}((n + m) \log n)$
Floyd	caminos más cortos entre todos los pares	matriz	$\Theta\big(n^3\big)$
Kruskal	árbol de expansión mínima	conjuntos disjuntos	$\mathcal{O}(m \log n)$